PROFESSIONAL SKILLS

Technical Writing & Documentation

BLUF, Diataxis, ADRs, design docs, runbooks, API references, handover packs and diagrams-as-code — with worked rewrites and templates you can copy.

technical-writing · documentation · bluf · diataxis · adr · design-docs · runbooks · readme · api-documentation · openapi · handover · diagrams-as-code · docs-as-code · style-guides · fde

Study Pack · Comprehensive reference & interview prep · ~121 min read · 26,557 words

CONTENTS

1. Why Writing Is the Highest-Leverage Engineering Skill
2. Audience, Purpose and the Cost of Ambiguity
3. BLUF and the Pyramid Principle
4. The Diátaxis Framework
5. Sentence-Level Craft
6. Structuring a Document
7. The README That Earns Its Keep
8. Architecture Decision Records
9. Design Documents and RFCs
10. Runbooks and Operational Documentation
11. API and Integration Documentation
12. The Handover Pack
13. Diagrams
14. Release Notes, Changelogs and User-Facing Comms
15. Email, Chat and Async Communication
16. Writing with AI Assistance
17. Docs as a Product
18. Interview Questions
19. Writing Drills and a 30-Day Improvement Plan

Why Writing Is the Highest-Leverage Engineering Skill

A Forward Deployed Engineer's code runs inside one customer's estate and is read by a handful of people. Their writing is read by everyone who decides, funds, operates or extends that work — and continues to be read after they have left the site. Writing is the only artefact that scales past your presence, survives your departure, and determines whether the thing you built is understood, trusted, adopted and maintained. It is not a soft skill adjacent to engineering; it is the mechanism by which engineering work produces durable value.

>50%
OF AN FDE'S OUTPUT, BY VALUE, IS PROSE

1 : 100
RATIO OF WRITERS TO READERS FOR A GOOD DOC

6 weeks
THE TEST: WHAT BREAKS IF YOU ARE UNREACHABLE

~15 s
HOW LONG AN EXECUTIVE GIVES YOUR DOCUMENT BEFORE DECIDING TO READ ON

The departure test

Everything in this guide reduces to one question: when you leave the engagement, what remains that works without you? Code remains, but code answers "what does this do", never "why is it like this", "what did we deliberately not do", "what happens when it breaks at 3 a.m.", or "who do I call". Those answers exist only in writing or not at all.

WRITING IS DECISION INFRASTRUCTURE

An unwritten decision gets re-litigated every quarter by whoever joins next. A written decision with its reasoning and its rejected alternatives ends that conversation in ninety seconds, permanently.

WRITING SCALES YOUR ATTENTION

You can explain the architecture to four people. A good architecture overview explains it to forty, including people who join next year, without consuming any of your time.

WRITING FORCES REAL THINKING

Vague understanding survives conversation and dies on the page. The act of writing a design doc routinely reveals that you do not know how the failure path works. That discovery is the point.

WRITING IS HOW YOU ARE EVALUATED

The people who renew contracts and decide promotions almost never see your code. They see your status reports, design docs and summaries. Fairly or not, that is the sample they judge from.

WRITING REDUCES SUPPORT LOAD

Every question answered in a doc is a question not asked in a Slack channel at 22:00. The runbook that prevents one incident call pays for itself ten times over.

WRITING IS THE HANDOVER

The handover pack is the most commercially valuable artefact an FDE produces. A system that cannot be operated from written material has not been delivered; it has been lent.

The asymmetry that justifies the effort

Writing a page well might cost you two hours instead of thirty minutes. That page will be read by thirty people over two years. If the clearer version saves each reader five minutes of confusion, the return is 150 minutes against 90 minutes invested — and that ignores the far larger cost of the decisions made wrongly because the unclear version was misread. Documentation economics almost always favour spending more time than feels natural.

What FDEs actually write

Artefact	Primary reader	Lifespan	Cost of getting it wrong
Weekly status	Sponsor	One week	Surprise, lost trust, late escalation
Discovery findings	Sponsor, steering	The engagement	Wrong problem solved for six months
Design doc / RFC	Engineers, architects, security	Years	Rework, a rejected design at the last gate
ADR	Future maintainers	Life of the system	The same decision argued every year
README	The next engineer	Life of the repo	Hours lost per newcomer, forever
Runbook	Operator at 3 a.m.	Life of the system	Extended outage, wrong remediation
API docs	Integrating developers	Life of the API	Support burden, wrong integrations shipped
Handover pack	The customer's team	Years	The system becomes unmaintainable; no reference
Release notes	Users, operators	Permanent record	Unannounced breaking change, lost confidence
Email / chat	Everyone, daily	Minutes to forever	Decisions missed, ambiguity compounding

Why engineers write badly, specifically

1. **The curse of knowledge.**
   You cannot un-know the context, so you cannot see which assumptions you have skipped. The fix is mechanical: give the draft to someone outside the project and watch where they stop.

2. **Writing in discovery order.**
   You narrate the journey — investigation, dead ends, eventual conclusion — because that is how you experienced it. Readers need the conclusion first and the journey optional.

3. **Precision anxiety.**

Fear of being wrong produces hedges: "may", "could potentially", "in certain circumstances". Three hedges in a sentence leave the reader unable to act. Be specific about conditions instead of vague about everything.

4 **Writing to impress rather than to transfer.**
Complexity signals effort; it does not signal competence. The highest-status writing in engineering is the clearest.

5 **Treating documentation as a phase.**
Docs written after the work, in a rush, from memory, are bad. Docs written during the work, as the thinking happens, are good and cost less.

6 **No audience model.**
Writing one document for executives, engineers and operators simultaneously produces something useless to all three.



Audience, Purpose and the Cost of Ambiguity

Before any sentence, two questions: who is reading this, and what do they need to do afterwards? Everything else — length, structure, vocabulary, level of detail — follows deterministically from those answers. Most bad technical documents are not badly written; they are correctly written for a reader who does not exist.

The reader's job-to-be-done

Reader	Their job after reading	Time they will give you	What they need
Executive sponsor	Approve, fund, or escalate	90 seconds	Decision, number, risk, date. No mechanism.
Engineering manager	Allocate people, sequence work	5 minutes	Scope, effort, dependencies, risk
Implementing engineer	Build the thing correctly	30+ minutes, revisited	Interfaces, edge cases, constraints, rationale
Reviewer (security, architecture)	Approve or reject against criteria	20 minutes	Their checklist, addressed line by line
Operator on call	Restore service now	60 seconds, under stress	Numbered steps, verification, rollback
Integrating developer	Make a successful API call	10 minutes to first success	A working example they can paste
Future maintainer	Change it safely	As long as it takes	Why, not what. Rejected options.
Auditor	Verify a control exists	Whatever is needed	Dates, owners, evidence, unambiguous statements

✦ **Write the reader at the top of your draft**

Literally type "READER: the on-call operator at 3 a.m. who has never seen this system" as the first line of the file, and delete it before publishing. It keeps you honest for the whole document, and it makes the "is this sentence necessary?" question answerable instead of a matter of taste.

The cost of ambiguity

TEXT

```
AMBIGUOUS                          WHAT A READER MIGHT DO
"The job runs nightly."            02:00? 23:59? Their timezone? Yours?
                                   → builds a downstream job at 01:00 UTC
                                      and gets stale data for a year

"Retry on failure."                Retry how many times? What backoff?
                                   Which failures - 500s only, or 429s too?
                                   → infinite retry storm during an outage

"Large files are not supported."   What is large? 10 MB? 1 GB?
                                   → user uploads 200 MB, silent truncation

"Should be fast enough."           For what? Measured how? At what load?
                                   → no SLO, no alerting, no recourse

"Contact support if issues."       Which support? What hours? What SLA?
                                   → ticket into a queue nobody reads

"The field is optional."           Optional to send, or nullable in storage?
                                   What happens downstream if absent?
                                   → NullPointerException in a batch job

UNAMBIGUOUS
"The job runs at 02:00 UTC daily. On failure it retries 3 times with
 exponential backoff (1 s, 4 s, 16 s) for HTTP 429 and 5xx only; 4xx
 other than 429 fail immediately and page the on-call rota.

 Files above 100 MB are rejected with HTTP 413 and a RequestTooLarge
 error; no partial processing occurs.

 p95 end-to-end latency target is 400 ms at up to 50 requests/second.

 omitted_field defaults to null in storage; downstream consumers must
 treat null as 'not provided', never as zero."
```

Same facts, three documents

The single most useful exercise in technical writing is taking one set of facts and rendering it for three audiences. Below is the same incident — a data pipeline failure that produced incorrect month-end figures — written three ways. The facts do not change. Everything else does.

VERSION 1 — FOR THE EXECUTIVE SPONSOR

TEXT

```
Subject: Month-end revenue figures were overstated by 2.1% - corrected,
         root cause fixed

The revenue figures in the 3 March month-end pack were overstated by
EUR 412,000 (2.1%). Corrected figures are attached. No external reporting
or customer invoicing used the incorrect numbers.

What happened: a currency-conversion step silently skipped 1,840 GBP
transactions on 1 March because an upstream rate feed returned an empty
response and our job treated that as "no transactions to convert" rather
than as a failure.

Impact: internal management reporting only, for 4 days, between the pack
being issued on 3 March and correction on 7 March. The Q1 board pack had
not yet been produced. Nothing was filed, invoiced or communicated
externally on these numbers.

What we have done: the job now fails loudly rather than continuing when
the rate feed is empty, and a daily reconciliation check compares
transaction counts before and after conversion. Both are live as of
6 March.

What we are still doing: reviewing the other eleven jobs in this pipeline
for the same pattern. Complete by 21 March. I will confirm.

Your decision needed: none. Flagging in case you want to inform the CFO
ahead of the Q1 pack, which I would recommend.

Arun
```

VERSION 2 — FOR THE ENGINEERING TEAM

TEXT

```
INCIDENT NOTE: INC-2026-0144
Silent data loss in fx_convert step; 1,840 GBP rows unconverted

SUMMARY
On 2026-03-01 the fx_convert stage of the revenue pipeline processed
0 of 1,840 GBP transactions, passing them through unconverted. The
resulting month-end aggregate overstated EUR revenue by EUR 412,000
(2.1%). Detected 2026-03-06 by a finance analyst, not by monitoring.

TIMELINE (all times UTC)
2026-03-01 02:14  rates_fetch job calls ECB endpoint, receives HTTP 200
                  with an empty body. Logged at INFO as "0 rates fetched".
2026-03-01 02:15  fx_convert loads the (empty) rate table, finds no rate
                  for GBP, and takes the `continue` branch in the loop.
                  Logs "skipped 1840 rows (no rate)" at DEBUG.
2026-03-01 02:40  aggregate_revenue runs over mixed-currency rows.
2026-03-03 09:00  Month-end pack issued with incorrect totals.
2026-03-06 11:20  Finance analyst queries a 2.1% variance vs their own
                  GBP ledger extract.
2026-03-06 14:05  Root cause confirmed.
2026-03-06 17:30  Fix deployed; pipeline re-run; corrected pack issued
                  2026-03-07 08:00.

ROOT CAUSE
Three failures compounded:
 1. rates_fetch treated HTTP 200 + empty body as success. The ECB
    endpoint returns 200 with an empty payload during its own
    maintenance window rather than a 5xx.
 2. fx_convert's "no rate for currency" path was a `continue` with a
    DEBUG log, i.e. designed to tolerate unknown currencies, with no
    distinction between "currency we don't care about" and "rate table
    is empty".
 3. No reconciliation between input row count and converted row count.

CONTRIBUTING
 - DEBUG level is not shipped to the log aggregator in prod, so the
   "skipped 1840 rows" line existed only on the pod and was lost on
   restart.
 - No alert on rate-table row count.

FIXES
 [DONE 2026-03-06] rates_fetch raises EmptyRateTable if the parsed rate
   count is 0; job exits non-zero; alert routes to the data on-call rota.
 [DONE 2026-03-06] fx_convert raises if skipped_rows / total_rows > 0.01,
   and logs skip counts at WARN with the currency breakdown.
 [DONE 2026-03-07] Daily reconciliation check: count(input) ==
   count(converted) + count(explicitly_excluded). Fails the DAG.
 [IN PROGRESS, due 2026-03-21] Audit the other 11 pipeline stages for
```

```
    the same "continue on missing reference data" pattern. Owner: Sana.
  [IN PROGRESS, due 2026-03-14] Ship WARN and above from all prod
    namespaces to the aggregator. Owner: platform team, ticket PLAT-2291.

LESSON
"Zero rows" is not a valid default for reference data. Any stage that
loads a lookup table must assert non-emptiness before use. We are adding
this to the pipeline design checklist.
```

VERSION 3 — FOR THE OPERATOR WHO RUNS THE PIPELINE

TEXT

```
RUNBOOK ENTRY: fx_convert skipped rows alert

ALERT NAME   fx_convert_skip_ratio_high
FIRES WHEN   More than 1% of rows in a fx_convert run are skipped for
             "no rate available"
SEVERITY     P2 - data correctness. Do not ignore. Downstream month-end
             figures will be wrong if this is not resolved before 02:40.

WHAT IT MEANS
The currency rate table was empty or missing entries when the conversion
ran. Rows passed through in their source currency and any aggregate built
on them will be wrong.

CHECK THIS FIRST (60 seconds)
 1. Run:  SELECT count(*) FROM fx_rates WHERE rate_date = CURRENT_DATE;
    Expected: 31 or more rows.
    If 0  → the rate fetch failed. Go to step 2.
    If >0 → a specific currency is missing. Go to step 4.

 2. Check the rates_fetch job:
    kubectl logs -n data job/rates-fetch --tail=50
    Look for "EmptyRateTable" or a non-200 response.

 3. Re-run the rate fetch, then the conversion:
    kubectl create job --from=cronjob/rates-fetch rates-fetch-manual
    Wait for completion, re-check the count in step 1, then:
    kubectl create job --from=cronjob/fx-convert fx-convert-manual

 4. If a single currency is missing, check whether it is new:
    SELECT DISTINCT currency FROM transactions
    WHERE txn_date = CURRENT_DATE
      AND currency NOT IN (SELECT currency FROM fx_rates
                           WHERE rate_date = CURRENT_DATE);
    If a new currency has appeared, this needs a data-team decision.
    Escalate - do not guess a rate.

VERIFY YOU HAVE FIXED IT
   SELECT count(*) FROM revenue_staging WHERE currency ≠ 'EUR';
   Expected: 0. If non-zero, conversion is still incomplete.

IF MONTH-END HAS ALREADY RUN (after 02:40)
   The aggregates are wrong. You must re-run aggregate_revenue after
   fixing the conversion, and tell the finance contact (Karin Weber,
   ext 4412) that the figures have changed. Do not wait until morning.

ROLLBACK
   None required. Re-running is idempotent; aggregates are rebuilt from
   source each run.
```

```
ESCALATE TO
   Data on-call rota (PagerDuty: data-platform)
   If unresolved by 06:00: Sana Ali, then Arun Krishnan.
```

✓ **What changed between the three**

The executive version answers "do I need to act, and is my reputation exposed?" in the first paragraph, and contains no mechanism at all. The engineering version is a timeline plus causes plus tracked fixes with owners and dates, and its most valuable line is the generalised lesson. The operator version is imperative, numbered, has exact commands, a verification step, and an explicit "what if it is already too late" branch. One document attempting all three would fail all three. Writing three documents took about ninety minutes total and each one is shorter than the merged version would have been.

Choosing the level of detail

- [ ] Cut any sentence whose removal does not change what the reader does.
- [ ] If a detail matters only to one reader group, move it to an appendix or a separate document and link it.
- [ ] Give the reader an exit: "if you only need X, stop after section 2."
- [ ] State the thing, then the reasoning — never the reasoning then the thing.
- [ ] Replace any internal-journey narrative ("we first tried…") with a rejected-options section if it matters, or delete it if it does not.
- [ ] Assume the reader will not read the next paragraph, and write accordingly.

BLUF and the Pyramid Principle

Bottom Line Up Front means the conclusion is the first thing on the page, followed by the support that justifies it, followed by the detail that justifies the support. It is the opposite of how investigation feels and the opposite of how academic writing is taught, and it is correct for essentially all professional engineering communication. The reader must be able to stop after any paragraph and still have the most important remaining information.



The pyramid: answer, then the reasons that make it true, then the evidence under each reason. Each level answers "why?" for the level above.

The rules

1. **Lead with the conclusion.**
   The first sentence states the answer, the decision needed, or the thing that happened. Not the context, not the background, not the process.

2. **Group supporting points MECE.**
   Three to five, non-overlapping, collectively covering the case. If you have nine points, you have not found the right grouping.

3. **Each group answers "why?" for the level above.**
   If a point does not support the conclusion, it belongs elsewhere or nowhere.

4. **Order by importance, not chronology.**
   Chronology is for timelines and runbooks only.

5. **Make every heading informative.**
   "Options considered" says nothing; "Managed Postgres costs 30% more but removes the DBA dependency" is itself an argument.

6. **Pass the skim test.**
   Read only the title, headings and first sentence of each paragraph. If that is not a coherent, complete summary, the structure is wrong.

Worked rewrite 1 — a status update

TEXT

```
BEFORE (280 words, conclusion in sentence 11)

Hi all,

This week has been quite busy. On Monday I continued working on the
ingestion service and spent some time looking at the schema that the
vendor provided, which turned out to be different from what we had been
told in the kickoff. I raised this with Tom and he said he would look
into it but hasn't got back to me yet. On Tuesday and Wednesday I was
mostly in meetings - the architecture review went reasonably well
although they had some questions about how we handle PII that I need to
follow up on. I also started looking at the test environment but the
credentials we were given don't seem to work, so I've raised a ticket
with IT (ticket number 88412) and am waiting to hear back. Thursday I
managed to get back to the ingestion work and made some progress on the
parsing logic, although the date handling is more complicated than
expected because there appear to be three different date formats in the
source files. I've handled two of them so far. Friday I spent mostly on
documentation and catching up on emails.

Overall I think we're broadly on track although the schema issue and
the environment access could become problems if they aren't resolved
soon. Let me know if you have any questions.

Arun
```

TEXT

```
AFTER (190 words, conclusion in sentence 1, two clear asks)

Subject: Ingestion - week 3 - AMBER - two blockers need your help

AMBER. Parsing is on track; two external blockers now have less than a
week of float. I need help with both.

BLOCKERS - I need you to act
 1. Test environment credentials do not work. Raised with IT as ticket
    88412 on Tuesday, no response. If not resolved by Wednesday,
    integration testing slips a week.
    ASK: Priya, can you chase at your level?
 2. The vendor's schema differs from what was agreed at kickoff -
    specifically, order_line is nested rather than flat. Raised with
    Tom on Monday, no answer yet. This changes 2 days of parsing work.
    ASK: who owns the vendor relationship? I need a decision by Thursday.

PROGRESS
 - Parsing logic handles 2 of 3 date formats found in the source files.
   The third (DD.MM.YY with 2-digit years) needs a century rule - I will
   assume 2000-2099 unless told otherwise.
 - Architecture review passed with one follow-up: document PII handling
   in the ingest path. Doing that Monday.

NEXT WEEK
 - Third date format, PII documentation, integration test harness.

Arun
```

What the rewrite did

It removed all chronology, which was carrying no information. It promoted the two blockers — the only parts requiring another person to act — above everything else and gave each an explicit named ask with a deadline. It converted "the date handling is more complicated than expected" into a specific, resolved decision with a stated default. It cut a third of the words and roughly tripled the information density. Notably, the "after" version is also easier to write, because the structure tells you what belongs where.

Worked rewrite 2 — a design proposal

TEXT

```
BEFORE

Proposal: Message Queue Introduction

Background
Our current system processes orders synchronously. When an order is
received by the API, it is validated, written to the database, and then
the notification service is called, and then the warehouse integration
is called. This has worked fine up to now but as volumes have grown we
have started to see some issues. The warehouse integration in particular
can be slow at times and when it is slow, the whole request is slow,
because everything happens in sequence. We have also seen a few cases
where the warehouse system was down and orders failed entirely even
though they had already been written to our database, which caused some
confusion. There are various ways we could address this. One option
would be to make the warehouse call asynchronous using a thread pool,
although this has some drawbacks around durability. Another would be to
introduce a message queue ...

[continues for 4 more paragraphs before stating the recommendation]
```

TEXT

```
AFTER

Proposal: Decouple warehouse integration with a durable queue
Recommendation: RabbitMQ (managed), 6 days of work, no new vendor.

RECOMMENDATION
Move the warehouse integration out of the synchronous order path and
behind a durable message queue. Use the managed RabbitMQ instance that
the platform team already operates. 6 days of work, no new vendor
onboarding, no new operational burden on us.

WHY NOW — three reasons
 1. Latency. Warehouse calls are p95 2.4 s, p99 11 s, and they sit
    inside the customer-facing request. Order API p99 is 12.3 s against
    a 2 s target, and the warehouse call accounts for 89% of it.
 2. Correctness. 34 orders in the last 90 days were written to our
    database and then failed at the warehouse step, leaving them in a
    state no process owns. Each required manual reconciliation.
 3. Coupling. The warehouse vendor's maintenance window (Sundays,
    02:00–06:00) currently takes our order API down with it.

OPTIONS CONSIDERED
 A. Thread pool + retry in-process
    Cheapest (2 days). Rejected: loses messages on pod restart, which
    is the exact failure mode in reason 2 above.
 B. Database-backed outbox + poller
    No new infrastructure (4 days). Viable. Rejected: we would be
    building queue semantics ourselves — visibility timeouts, DLQ,
    poison-message handling — and the platform team already runs a queue.
 C. Managed RabbitMQ  [RECOMMENDED]
    6 days. Durable, already operated by platform, already in the
    security-approved list, existing monitoring and alerting.
 D. Kafka
    Rejected: 3 weeks, and we need work-queue semantics, not a log.
    Ordering guarantees and replay are not requirements here.

WHAT CHANGES
 Order API writes the order, publishes an OrderPlaced message, returns
 201. A consumer handles the warehouse call with retry and a DLQ.
 Expected Order API p99: 320 ms.

RISKS
 R1 Eventual consistency: the warehouse may not know about an order for
    up to ~5 s normally, longer during an outage. The ops team confirmed
    this is acceptable; their own pick waves run every 15 minutes.
 R2 DLQ needs an owner and a runbook. Covered in the rollout section.

ROLLOUT
```

```
Dual-write behind a feature flag for 1 week, compare, then cut over.
Rollback is a flag flip.
```

Worked rewrite 3 — an incident note

TEXT

```
BEFORE
"There was an issue with the API this morning. It started around 9am
and we noticed it when a few customers complained. We looked into it
and found that the database connections were being exhausted. We
restarted the service and it seems to be okay now. We'll keep an eye
on it."

AFTER
"INC-2026-0151: Order API returned 503 for 38 minutes this morning
(08:52–09:30 UTC), affecting an estimated 1,400 requests and at least
6 customers who contacted support.

Cause: the connection pool (max 20) was exhausted by a reporting query
introduced in yesterday's release that holds a connection for ~90 s.
Under normal morning load that consumed the entire pool within minutes.

Resolved by: restarting the service (immediate) and reverting the
reporting query (09:48). Both complete.

Still open: the restart is a mitigation, not a fix. Two actions –
(a) move reporting queries to the read replica, owner Sana, due 14 Mar;
(b) add an alert on pool utilisation above 80%, owner me, due 11 Mar.
Without (b) we would not have detected this before customers did.

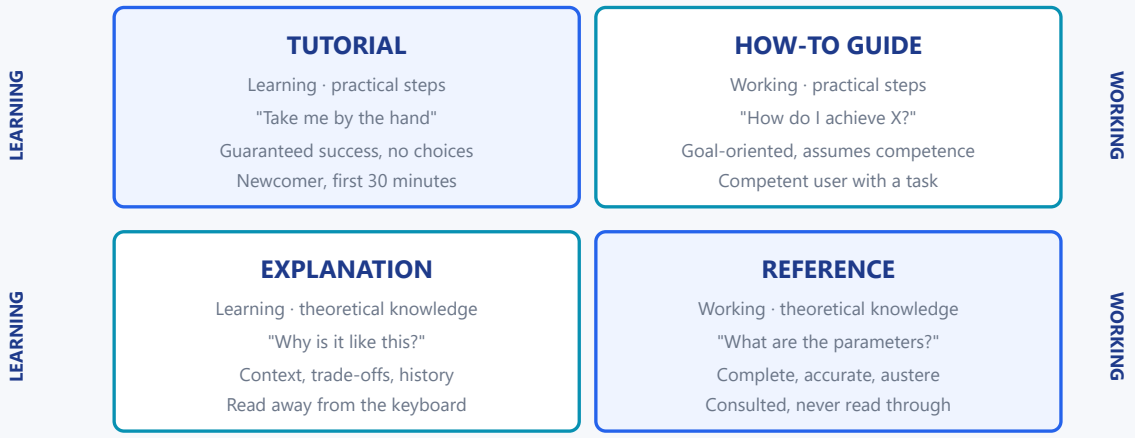
Not yet known: whether any of the 1,400 failed requests resulted in a
lost order. Checking against the client retry logs; will confirm today."
```

⚠ **"Seems to be okay now" and "we'll keep an eye on it"**

Two phrases that should never survive an edit. The first means you do not know whether it is fixed; say what you verified and how. The second means no action is owned; replace it with a named person, a specific check and a date. Vague reassurance is worse than an honest unknown, because it stops anyone else from investigating.

The Diátaxis Framework

Most documentation is bad not because the sentences are poor but because four fundamentally different kinds of document have been blended into one. Diátaxis (sometimes called the Divio system) separates them by what the reader is doing: learning or working, and studying or applying. Once you internalise the four quadrants, the question "where does this paragraph go?" has an answer instead of being a matter of taste.



Four quadrants, four reader states. A document that tries to occupy two of them serves neither well.

What belongs where

	Tutorial	How-to	Reference	Explanation
Reader is	A beginner	Competent, has a goal	Working, needs a fact	Curious, wants understanding
Form	A lesson	A recipe	A map	An essay
Success	They finish and feel capable	They achieved the goal	They found the fact in 20 s	They can reason about new cases
Choices offered	None	Some, with guidance	All, listed	Discussed
Explains why	No — it interrupts	Briefly, where a choice is made	No	That is its entire purpose
Tested how	A newcomer completes it unaided	Steps work for the stated goal	Matches the implementation	A reader can answer a novel question
Length	30 min maximum	As short as possible	Exhaustive	As long as the idea needs

The failure modes of mixing

TEXT

```
FAILURE 1: TUTORIAL CONTAMINATED WITH EXPLANATION
"Now run `app init`. This uses the Raft consensus protocol under the
 hood, which was chosen because of its understandability compared to
 Paxos. Note that in a multi-region setup you would instead want ... "

 WHY IT FAILS: the beginner now has three new concepts, two of which
 are irrelevant to the task, and has lost the thread. Tutorials must
 be ruthlessly linear. Move all of that to an Explanation page and
 link it at the END.

 FIX: "Run `app init`. You should see 'Initialised in ./data'.
       (Curious about how clustering works? See Architecture.)"

FAILURE 2: HOW-TO THAT IS SECRETLY A TUTORIAL
 "How to configure SSO" that begins by explaining what SAML is, then
 sets up a local dev environment, then installs dependencies.

 WHY IT FAILS: the reader already has a running system and a specific
 goal. Making them wade through setup is why they give up and ask you
 in Slack instead.

 FIX: start with prerequisites as a checklist, then go straight to
 the steps that achieve the stated goal.

FAILURE 3: REFERENCE WITH OPINIONS
 "timeout - the request timeout in seconds. We recommend 30 for most
  use cases, although in high-latency environments you may want ... "

 WHY IT FAILS: reference must be scannable and complete. Recommendations
 belong in a how-to. Mixed in, they bloat the table and age badly.

 FIX: reference says: "timeout | integer | seconds | default 30 |
      max 300 | applies per attempt, not per request including retries"
      The recommendation goes in "How to tune for high-latency networks".

FAILURE 4: EXPLANATION PRETENDING TO BE REFERENCE
 An "Architecture" page that is actually a narrative of how the system
 evolved, with no diagram and no component list.

 WHY IT FAILS: someone looking for "what talks to what" cannot find it.

 FIX: split. A reference-style component and interface list, plus an
 explanation of why the boundaries are where they are.

FAILURE 5: THE EVERYTHING PAGE
 A single README containing a quickstart, full config reference, design
 rationale, troubleshooting and contribution guidelines, at 4,000 words.
```

```
WHY IT FAILS: nobody can find anything, and nobody maintains it
because every change touches a 4,000-word file.

FIX: README is a map plus a quickstart. Everything else is linked.
```

Applying it to an FDE's artefacts

Artefact	Quadrant	Common mistake
README quickstart	Tutorial	Adding configuration options; it must have exactly one path
"How to add a new data source"	How-to	Starting with architecture background
Runbook	How-to (under stress)	Explaining root cause before the remediation steps
API endpoint list	Reference	Inconsistent field documentation; missing error cases
Config options table	Reference	Opinions and recommendations mixed in
ADR	Explanation	Describing the implementation instead of the reasoning
Design doc	Explanation + reference	Omitting rejected options, which is the valuable part
Handover architecture overview	Explanation	Being a diagram with no prose, or prose with no diagram

✦ **The question that resolves every placement dispute**
"Is the reader learning or working, and do they need to acquire knowledge or apply it?" Two binary questions, four answers, one quadrant. If a paragraph does not fit the quadrant of the page it is on, it is not a bad paragraph — it is a paragraph in the wrong file. Move it and link to it.

Sentence-Level Craft

Structure gets a reader to the right paragraph; sentences determine whether they understand it. Technical prose has one job — to transfer a precise idea with minimum resistance — and almost all of the resistance comes from four habits: the passive voice hiding the actor, nominalisations turning verbs into fog, hedges removing all actionable content, and imprecision about numbers and conditions.

Plain language

Instead of	Write
utilise, leverage	use
in order to	to
at this point in time	now
in the event that	if
due to the fact that	because
has the capability to	can
a number of	the actual number
facilitate	help, or name the actual action
prior to / subsequent to	before / after
it is recommended that you	use the imperative: "Set X to Y"
perform a validation of	validate
provide assistance to	help
in terms of	(usually deletable)
on a daily basis	daily
functionality	feature, or name the thing it does

> **Plain is not simplistic**
> Plain language means removing friction, not removing precision. "The consumer acknowledges the message after the write commits" is plain and precise. "The message acknowledgement is performed subsequent to the completion of the write transaction" is neither clearer nor more rigorous — it is the same claim with more syllables and a hidden actor.

Active voice and the hidden actor

The passive voice is not wrong; it is a tool that hides the actor, which is occasionally what you want and usually not. In technical writing the actor is almost always the information the reader needs.

TEXT

```
PASSIVE, ACTOR HIDDEN              ACTIVE, ACTOR NAMED
"The record is validated."    →    "The ingest service validates the record."
                                   (Now you know where to look when it fails.)

"Permissions must be granted." →  "The workspace admin grants permissions
                                    in Settings > Access."
                                   (Now the reader knows who to ask.)

"It was decided that ... "         → "The architecture board decided on
                                    11 March that ... "
                                   (Accountability and a date.)

"Mistakes were made."              → "I submitted the security design a week
                                    late."

LEGITIMATE PASSIVE - when the actor is unknown, irrelevant, or obvious:
 "The file is deleted after 30 days."   (by the retention job; fine)
 "Requests are rate-limited to 100/min." (by the gateway; fine in reference)
 "The server was compromised on 3 March." (actor genuinely unknown)
```

Nominalisations

TEXT

```
A nominalisation is a verb wearing a noun costume. It adds words,
removes the actor, and makes the sentence harder to picture.

"We performed an analysis of the logs"
  → "We analysed the logs"

"The implementation of the change was carried out by the platform team"
  → "The platform team implemented the change"

"Provision of access is dependent upon completion of the security review"
  → "You get access after the security review passes"

"There is a requirement for the validation of input parameters"
  → "Validate input parameters"

"Our recommendation is the utilisation of the managed service"
  → "We recommend the managed service"

SPOT THEM: words ending in -tion, -ment, -ance, -ity, -al preceded by a
weak verb (perform, conduct, carry out, undertake, provide, make, do,
achieve, is/are). Turn the noun back into the verb and the sentence
usually loses a third of its words.
```

Cutting hedges

TEXT

```
HEDGED (unactionable)
"It may be possible that in certain circumstances the job could
 potentially take somewhat longer than usual to complete."

WHAT THE READER NEEDS
"The job takes 4–6 minutes normally. If the source file exceeds 2 GB it
 takes up to 25 minutes."

THE RULE: replace vagueness about everything with precision about
conditions. Uncertainty is fine — state its shape.

BAD UNCERTAINTY   "Performance might be affected."
GOOD UNCERTAINTY  "We have not load-tested above 200 rps. Below that,
                   p95 is 180 ms. Above it, we do not know."

BAD UNCERTAINTY   "This could be a problem."
GOOD UNCERTAINTY  "If the rate feed is empty we would silently skip
                   rows. I have not confirmed whether the feed can
                   return empty; checking today."

HEDGES TO DELETE ON SIGHT
 basically, essentially, actually, really, quite, fairly, somewhat,
 rather, arguably, I think, it seems, sort of, kind of, in a sense,
 to some extent, as it were, generally speaking

HEDGES TO KEEP (they carry real meaning)
 "approximately 40%" when you have not measured exactly
 "we expect" when stating a prediction rather than a fact
 "in our testing" when scoping a claim to its evidence
```

Precision about numbers and conditions

Imprecise	Precise	Why it matters
"fast"	"p95 under 200 ms at 100 rps"	Fast is unverifiable; a number is a contract
"most users"	"the 1,840 users with an admin role"	"Most" cannot be tested or scoped
"improved significantly"	"26.1 h to 5.2 h median, same query"	Significant means nothing without the before
"a few retries"	"3 retries at 1 s, 4 s, 16 s"	The implementer will otherwise guess
"regularly"	"every 15 minutes, on the quarter hour"	Scheduling bugs live here
"soon"	"by 14 March"	Soon is whatever the reader hopes
"should work"	"tested on Ubuntu 22.04 and 24.04; not tested on Windows"	Scope the claim to the evidence
"the latest version"	"version 4.2.1 or later"	Latest changes meaning over time
"large files"	"files above 100 MB"	The boundary is the whole point
"and/or"	Say which, or say "one or both"	Ambiguity in a spec becomes a bug

Parallelism

TEXT

```
NOT PARALLEL (the reader's brain has to re-parse each item)
 The service:
  - validates incoming records
  - transformation of the payload
  - it then writes to the database
  - notification

PARALLEL (same grammatical shape, scannable)
 The service:
  - validates incoming records
  - transforms the payload
  - writes to the database
  - notifies the warehouse integration

PARALLEL HEADINGS
 BAD                          GOOD
  Installing                   Install the CLI
  Configuration                Configure credentials
  How to run it                Run your first job
  Troubleshooting              Troubleshoot common failures

PARALLEL TABLE COLUMNS
 Every cell in a column should answer the same question in the same form.
 If one cell is a sentence and the next is a fragment, fix it.
```

Fifteen worked before/after pairs

TEXT

```
1. BEFORE  The functionality for the purposes of user authentication has
           been implemented.
   AFTER   Users can now sign in.
   WHY     Nominalisation, passive, and the reader cares about the
           capability, not that it was implemented.

2. BEFORE  It should be noted that the API has a rate limit.
   AFTER   The API allows 100 requests per minute per API key. Requests
           above that return HTTP 429 with a Retry-After header.
   WHY     "It should be noted" is pure filler; the useful content
           (the actual numbers and behaviour) was missing.

3. BEFORE  Due to the fact that the database was experiencing high load,
           response times were impacted.
   AFTER   Database load pushed p95 response time from 180 ms to 4.2 s
           between 09:00 and 09:38.
   WHY     "Impacted" hides direction and magnitude; now it is measurable.

4. BEFORE  The user is required to provide their credentials prior to
           being able to access the system.
   AFTER   Sign in before using the system.
   WHY     19 words to 6, imperative, same meaning.

5. BEFORE  We have identified a number of potential issues that may
           possibly impact the timeline.
   AFTER   Three things could delay us: the vendor schema change (2 days),
           test environment access (up to 1 week), and the pen test
           booking (unknown, chasing).
   WHY     Replaced hedging with a count, a list and sized impacts.

6. BEFORE  Error handling has been added to the parser.
   AFTER   The parser now returns ParseError with the line number and the
           offending token instead of raising an unhandled exception.
   WHY     "Error handling has been added" tells a reviewer nothing about
           what behaviour changed.

7. BEFORE  This document aims to provide an overview of the architecture
           of the solution that has been developed.
   AFTER   This document describes how the order pipeline works and why
           the warehouse integration is asynchronous.
   WHY     Deleted the throat-clearing; stated the two actual topics.

8. BEFORE  In the event that the connection fails, a retry will be
           attempted.
   AFTER   If the connection fails, the client retries 3 times with
           exponential backoff (1 s, 4 s, 16 s), then raises
           ConnectionExhausted.
   WHY     "A retry will be attempted" leaves every useful parameter out.
```

```
9. BEFORE  Significant improvements were made to the performance of the
           ingestion process.
   AFTER   Ingestion of a 1 GB file went from 42 minutes to 6 minutes,
           measured on the same hardware with the same test file.
   WHY     Named the measurement and the conditions, so it is checkable.

10. BEFORE The team should consider whether it might be appropriate to
           potentially migrate to a managed service at some point.
    AFTER  I recommend migrating to managed Postgres in Q3. It removes
           our DBA dependency and costs EUR 340/month more.
    WHY    Four hedges removed; an owner, a recommendation, a date and
           a cost added.

11. BEFORE Configuration of the system is performed through the use of
           environment variables.
    AFTER  Configure the system with environment variables.
    WHY    Two nominalisations and a passive in a nine-word idea.

12. BEFORE There is an issue with the way that dates are being handled
           in certain cases.
    AFTER  Dates in DD.MM.YY format with two-digit years are parsed as
           19xx instead of 20xx. This affects 1,840 records from the
           Hamburg source file.
    WHY    Named the exact defect, the exact input and the blast radius.

13. BEFORE Please find attached the documentation for your review.
    AFTER  Attached is the design doc. I need your decision on the queue
           choice (section 4) by Thursday; the rest is context.
    WHY    Told the reader what to do, where to look and by when.

14. BEFORE The system has been designed with scalability in mind.
    AFTER  The consumer scales horizontally: tested to 8 replicas at
           1,200 messages/second with no reordering issues. Beyond that
           the database write becomes the bottleneck.
    WHY    "Designed with X in mind" is a claim with no evidence; the
           rewrite states the tested limit and the next constraint.

15. BEFORE We weren't able to complete the migration because of various
           blockers that came up.
    AFTER  The migration is not complete. Two blockers: the read-replica
           lag exceeded 40 s during the dual-write test (unresolved,
           owner me, investigating Monday), and the DBA sign-off slot
           moved to 18 March. New target: 22 March.
    WHY    Replaced a vague excuse with named causes, owners and a date.
```

The ruthless editing checklist

Run this on every document before you publish. It takes ten minutes and typically removes 20–30% of the words.

- ☐ Is the conclusion in the first two sentences?
- ☐ Does every heading say something, rather than naming a category?
- ☐ Read only the headings and first sentences — is that a complete summary?
- ☐ Delete every instance of: basically, essentially, actually, really, quite, very, in order to, it should be noted, there is/are (where a real verb is hiding).
- ☐ Find every "is", "are", "was", "were" — is there a hidden passive or nominalisation?
- ☐ Find every vague quantifier — some, many, most, few, several — and replace with a number or justify it.
- ☐ Find every "should", "may", "could" — is it a real conditional or a hedge?
- ☐ Does every number have a unit, a basis and a source?
- ☐ Does every action have an owner and a date?
- ☐ Are list items grammatically parallel?
- ☐ Is any sentence longer than 35 words? Split it.
- ☐ Is any paragraph longer than 6 lines? Split it or make it a list.
- ☐ Is there a sentence whose removal would not change what the reader does? Remove it.
- ☐ Does any acronym appear before it is expanded?
- ☐ Would someone outside this project understand the first paragraph?

✦ **The 30% rule**

Write the draft, then cut 30% without losing information. It is almost always possible, and the result is almost always better. The cuts come from hedges, throat-clearing openings, restatements of the previous paragraph, and sentences that explain something the reader already understood. If you cannot find 30%, you have probably under-written rather than over-written — which is a different and rarer problem.

Structuring a Document

Readers do not read documents; they scan them, drop into the part that looks relevant, and read a few paragraphs. Structure is therefore not decoration but navigation, and a document with uninformative headings and uniform paragraph blocks is functionally unreadable regardless of the quality of its prose.

Headings that carry information

Category heading (weak)	Informative heading (strong)
Background	Why order latency became a problem in January
Options	Three ways to decouple the warehouse call
Option 3	Option 3: managed RabbitMQ — 6 days, no new vendor
Risks	Risks: eventual consistency and DLQ ownership
Conclusion	Recommendation: RabbitMQ, starting 14 March
Performance	Performance: p95 320 ms at 100 rps, degrades above 400
Troubleshooting	If the job fails with EmptyRateTable
Next steps	Next: Priya decides by Thursday, build starts Monday

> **The table-of-contents test**
> Generate the TOC from your headings and read it on its own. If it reads as a coherent argument or a complete map, the structure works. If it reads as a list of nouns — Background, Scope, Approach, Options, Risks, Conclusion — you have a document whose shape conveys nothing, and every reader will have to read the body to find out what you are proposing.

Chunking and progressive disclosure

1. **One idea per paragraph.**
   If a paragraph contains two ideas, the second one is invisible. Split it.
2. **Lead each paragraph with its point.**

The first sentence is the claim; the rest supports it. This makes the document skimmable by first sentences alone.

3. **Summary first, detail behind a link or a fold.**
   A reader who needs the detail will click; a reader who does not should never have to scroll past it.

4. **Use collapsible sections for reference material inside a narrative document.**
   Full config dumps, long error lists, SQL — present only when wanted.

5. **Put the thing people need most at the top, even if it is logically last.**
   Document order is a service to the reader, not a reflection of the problem's structure.

6. **Keep nesting to three levels.**
   A fourth level of heading means the document should be split.

Lists versus prose versus tables

Use	When	Avoid when
Prose	The ideas are connected by reasoning; causation matters	You are enumerating parallel items
Bulleted list	Parallel items with no inherent order	Items are actually sequential steps — use numbers
Numbered list	Order matters: steps, ranked priorities	Order does not matter — numbers imply it does
Table	Items share dimensions you want compared	Cells contain paragraphs — that is prose in a grid
Definition list	Term plus meaning, scanned by term	The terms need argument, not definition
Code block	Anything to be typed, pasted or matched exactly	Narrative explanation of code — put that in prose

> ⚠ **Bullet abuse**
>
> A document that is 80% bullets has usually had its reasoning removed. Bullets strip the connective tissue — because, therefore, but, unless — which is where the actual argument lives. If you find yourself writing nested bullets three levels deep, you are writing an outline of a document you have not written. Convert the top level back into prose paragraphs and keep bullets for genuinely parallel enumerations.

A general document skeleton

TEXT

```
TITLE: a sentence, not a noun phrase
  Bad:  "Order Pipeline Architecture"
  Good: "Order pipeline: why the warehouse call is asynchronous"

STATUS BLOCK (4 lines, always at the top)
  Status:    Draft | In review | Accepted | Superseded by [link]
  Owner:     Named human
  Updated:   2026-03-14
  Audience:  Engineers maintaining the order pipeline

TL;DR (3–5 lines, written last, read first)
  What this is, the conclusion, and what the reader should do.

CONTEXT (short)
  Only what the reader needs to understand the rest. Link out for more.
  If this section is longer than the recommendation, cut it.

THE SUBSTANCE
  Organised by argument, not chronology. Informative headings.
  Most important first.

WHAT THIS IS NOT / OUT OF SCOPE
  The most-skipped and most-valuable section in any document.

OPEN QUESTIONS
  Explicit, with owners. A document with no open questions is either
  finished or dishonest.

APPENDICES
  Full data, long tables, reproduction steps, raw output.

CHANGE LOG
  Date, who, what changed and why. Three lines per entry maximum.
```

Scannability mechanics

- [ ] Bold the two or three phrases per page that a skimmer must not miss — and no more, or bolding stops meaning anything.
- [ ] Keep paragraphs to 3–6 lines on screen; whitespace is a navigation aid, not wasted space.
- [ ] Put numbers in tables, not in sentences, when there are more than three of them.
- [ ] Use a consistent vocabulary: pick "job" or "task" and never alternate, even for variety. Elegant variation is a bug in technical prose.
- [ ] Front-load list items with the distinguishing word: "Timeout errors — …" not "If you see an error relating to timeouts, …".
- [ ] Never make the reader scroll back to understand a sentence; repeat a short definition rather than relying on memory.
- [ ] Give every document a stable anchor-able heading structure so people can link to a specific section in chat.

The README That Earns Its Keep

The README is the single highest-traffic document in any repository and usually the worst. Its job is narrow: tell a newcomer what this is, prove it works on their machine within ten minutes, and point them at everything else. A README that attempts to be complete reference material fails at all three. For an FDE, the README is also the first thing the customer's engineers will judge the handover by.

The structure

TEXT

````
═══════════════ README.md — annotated template ═══════════════

# order-pipeline

One sentence: what this is and who it is for.
"Ingests order files from the SFTP drop, normalises them, and publishes
OrderPlaced events to RabbitMQ. Operated by the Nordstahl data team."

>> ANNOTATION: no "welcome to", no project history, no logo. A reader who
   landed here from a search result must know within 5 seconds whether
   this is the right repo.

| | |
|---|---|
| Status | Production since 2026-03-28 |
| Owner | Nordstahl data team (#data-platform) |
| On call | PagerDuty rota `data-platform` |
| Runbooks | [docs/runbooks/](docs/runbooks/) |
| Design | [docs/design.md](docs/design.md) |

>> ANNOTATION: this table answers "who do I ask" and "where is the rest",
   which are the two most common reasons people open a README at all.

## What it does

Three to five sentences, or a short bullet list, plus one diagram.
Include the boundaries: what it does NOT do.

"It does not transform currency (see fx-service) and does not write to
SAP (see warehouse-bridge)."

>> ANNOTATION: stating what it is not prevents the most expensive kind of
   misunderstanding — someone building on a false assumption about scope.

## Quickstart

Prerequisites — exact versions, as a checklist:
- Python 3.11 or 3.12 (3.13 not supported — see issue #142)
- Docker 24+
- Access to the `dev` SFTP drop (request via #data-platform)

Then:

```bash
git clone git@github.com:nordstahl/order-pipeline.git
cd order-pipeline
make dev-up          # starts postgres + rabbitmq in docker
make install         # creates .venv and installs deps
````
````

````
make seed            # loads 50 sample orders
make run
```

You should see:

```
[INFO] watching /data/inbox
[INFO] processed sample-orders-01.csv: 50 records, 0 errors
[INFO] published 50 OrderPlaced events
```

If you do not, see [Troubleshooting](#troubleshooting).

>> ANNOTATION: the three things that make a quickstart actually work:
   (1) exact prerequisite versions including known-bad ones,
   (2) copy-pasteable commands with no placeholders to fill in,
   (3) the expected output, so the reader can self-diagnose.
   A quickstart that ends at "run the app" without showing what success
   looks like will generate a support question from every newcomer.

## Configuration

Only the variables needed to run it. Everything else links to reference.

| Variable | Required | Default | Notes |
|---|---|---|---|
| `SFTP_HOST` | yes | - | Use `localhost:2222` for local dev |
| `DB_URL` | yes | - | `postgresql:// ...`; `make dev-up` sets this |
| `BATCH_SIZE` | no | 500 | Rows per transaction; above 2000 risks timeout |
| `LOG_LEVEL` | no | INFO | DEBUG is very verbose; do not use in prod |

Full reference: [docs/configuration.md](docs/configuration.md)

>> ANNOTATION: README config section covers what you need for the
   quickstart to work. The 40 other variables go in reference docs.
   Mixing them is the most common README failure.

## How it works

One diagram and five sentences. Link to the design doc for depth.

>> ANNOTATION: resist writing the architecture here. The README's job is
   to orient, and the architecture will drift out of date in a file that
   nobody reviews.

## Development

- Run tests: `make test` (unit, ~20 s) / `make test-integration` (~4 min,
````
````

```
  needs `make dev-up`)
- Lint and format: `make lint` - CI will reject unformatted code
- Branch naming: `feat/`, `fix/`, `chore/` + ticket id
- Commits: conventional commits; the changelog is generated from them

## Troubleshooting

**`make dev-up` fails with "port 5432 already in use"**
You have a local Postgres running. Either stop it, or set
`POSTGRES_PORT=5433` before running.

**`make seed` fails with "relation orders does not exist"**
Migrations did not run. `make migrate` then retry.

**No events appear in RabbitMQ**
Check the management UI at http://localhost:15672 (guest/guest). If the
`orders` exchange is missing, `make dev-up` did not complete. Check
`docker compose logs rabbitmq`.

**Processing stops silently after ~500 records**
You are hitting the batch timeout. Reduce `BATCH_SIZE`.

>> ANNOTATION: this section should be grown from real questions. Every
   time someone asks you something in chat that is answerable here, add
   it - verbatim question, direct answer. Within a month it becomes the
   most valuable part of the file.

## Links

- [Design doc](docs/design.md) - why it is built this way
- [Runbooks](docs/runbooks/) - what to do when it breaks
- [ADRs](docs/adr/) - decisions and their reasoning
- [API reference](docs/api.md)
- [Handover pack](docs/handover.md)

## Licence / ownership
═══════════════════════════════════════════════════════════════
```

✕ **The quickstart must actually work, on a clean machine**

The most common README defect is a quickstart that works only for the author, because it depends on a tool they installed two years ago, an environment variable in their shell profile, or SSH access they forgot they had. Test it quarterly in a fresh container or on a new joiner's laptop, with no shortcuts, and fix whatever breaks. A broken quickstart costs every future reader thirty minutes and teaches them not to trust the rest of the documentation.

README anti-patterns

Anti-pattern	Why it hurts	Fix
Badge wall	Eight badges push the actual content below the fold	Keep build status and version; delete the rest
History and motivation first	Readers want to know what it does, now	Move to the design doc; link it
Full config reference inline	Buries the quickstart; goes stale	Table of the essential five; link the rest
Placeholders in commands	`export DB_URL=<your-db-url>` — the reader does not know what a valid one looks like	Give a working example value
No expected output	Reader cannot tell success from silent failure	Show the first lines of successful output
Stale screenshots	Actively misleading; worse than nothing	Date them, or avoid screenshots of UI you control
"Coming soon"	It never comes and the reader stops trusting the file	Delete, or link a ticket with a date
No owner	The reader has nobody to ask	Name a team and a channel at the top

The README test

TEXT

```
THE TEST: hand the repo to an engineer who has never seen it. Say
nothing. Watch.

MEASURE
 - Time from clone to first successful run
 - Number of times they have to ask you something
 - Number of times they open a file other than the README to proceed

PASS
 - Under 15 minutes, zero questions, zero other files needed

EVERY QUESTION THEY ASK IS A README DEFECT. Write the question down
verbatim, answer it in the README in the reader's words, and re-run the
test with the next newcomer. Three iterations of this produces a README
better than anything you can write by introspection.
```

Architecture Decision Records

An ADR captures one decision, its context, and its consequences, at the moment it is made. It exists because the most expensive missing information in any inherited system is *why* — why this database, why this boundary, why this odd-looking constraint that everyone is now afraid to remove. An FDE who leaves thirty ADRs behind has given the customer something more durable than the code.

Why ADRs, specifically

THEY STOP RE-LITIGATION

Without a record, every new senior engineer re-opens the same three debates. With one, the conversation is ninety seconds: here is what we knew, here is what we chose, here is what would change the answer.

THEY CAPTURE WHAT WAS REJECTED

The rejected options are usually more valuable than the chosen one, because they encode the constraints. "We rejected Kafka because we needed work-queue semantics, not a log" is a sentence that saves a future team two weeks.

THEY ARE CHEAP

A good ADR is one page and takes twenty minutes while the decision is fresh. Reconstructing the same reasoning a year later takes a day and produces a worse result.

THEY ARE HONEST ABOUT UNCERTAINTY

An ADR can say "we are not sure; we chose the reversible option." That is impossible to express in code and invaluable to the person who inherits it.

THEY DATE THE CONTEXT

Decisions are correct relative to what was known at a point in time. An ADR makes that explicit, so a later reversal is an update rather than an accusation.

THEY SURVIVE YOU

When the FDE leaves and the two engineers who were in the room move on, the ADR directory is the only remaining record of the system's reasoning.

The format

MARKDOWN

```
# ADR-0014: Use a managed message queue for warehouse integration

- Status: Accepted
- Date: 2026-03-11
- Deciders: A. Krishnan (FDE), T. Berg (enterprise architect), S. Ali
- Consulted: platform team, warehouse ops lead
- Supersedes: -
- Superseded by: -

## Context

The Order API calls the warehouse system synchronously inside the
customer request. Measured over 90 days:

- Warehouse call p95 is 2.4 s, p99 is 11 s
- Order API p99 is 12.3 s against a 2 s target; the warehouse call is
  89% of it
- 34 orders were written to our database and then failed at the
  warehouse step, leaving them in a state no process owns
- The warehouse vendor's Sunday maintenance window (02:00-06:00) takes
  our order API down with it

Constraints at the time of this decision:

- The platform team already operates a managed RabbitMQ cluster, which
  is on the security-approved list
- We have 6 weeks remaining in the engagement
- The customer has no Kafka operational experience and no appetite to
  acquire it during this engagement
- Eventual consistency of up to ~60 s is acceptable to warehouse ops,
  confirmed by M. Hoeffner on 2026-03-06 (their pick waves run every
  15 minutes)

## Decision

Publish an `OrderPlaced` message to the existing managed RabbitMQ
cluster from the Order API, and handle the warehouse call in a separate
consumer with retries and a dead-letter queue.

## Options considered

### A. Thread pool and in-process retry
2 days. Rejected: messages are lost on pod restart, which is exactly
the failure mode described in the context. It would make the system
look more reliable while leaving the correctness problem intact.

### B. Database-backed outbox with a poller
4 days, no new infrastructure. Genuinely viable. Rejected because we
```

```
would be implementing queue semantics ourselves - visibility timeouts,
dead-lettering, poison-message handling, backpressure - and the
platform team already operates a queue that provides them.

If RabbitMQ were not already available, this would have been the choice.

### C. Managed RabbitMQ  [CHOSEN]
6 days. Durable, already operated and monitored by the platform team,
already security-approved, and the team has existing runbooks for it.

### D. Kafka
Rejected. ~3 weeks including operational onboarding. We need work-queue
semantics with competing consumers, not an ordered replayable log.
Neither ordering across partitions nor replay is a requirement. The
operational cost is not justified by any requirement we have.

## Consequences

Positive:
- Order API p99 expected to drop from 12.3 s to ~320 ms
- Warehouse outages no longer affect order acceptance
- Failed warehouse calls land in a DLQ that can be inspected and
  replayed, rather than vanishing

Negative:
- The warehouse may not know about an order for up to ~60 s, and longer
  during an outage. Order status in the UI must reflect this; see
  ADR-0015.
- The DLQ needs an owner, an alert and a runbook. Without these it
  becomes a silent data loss mechanism. Covered by runbook RB-009.
- One more piece of infrastructure in the request path, albeit one the
  customer already runs.

Neutral:
- Testing becomes asynchronous, which requires a different integration
  test approach (see test/integration/README.md).

## What would change this decision

- If message volume exceeded ~5,000/s sustained, or if replay of the
  full order history became a requirement, Kafka would be worth
  re-evaluating.
- If the platform team stopped operating RabbitMQ, option B becomes the
  fallback and is about 4 days of work.
```

A worked ADR that rejects everything

MARKDOWN

```
# ADR-0021: Do not build a customer-facing status portal

- Status: Accepted
- Date: 2026-04-02
- Deciders: P. Menon (sponsor), A. Krishnan
- Consulted: CS team lead, 3 CS agents, 2 key-account managers

## Context

Customers email to ask "where is my order?" - 4,210 messages in the
Jan 2026 baseline, 31% of all inbound volume and 21% of agent minutes.
The obvious response is a self-service status portal.

Before building one we checked three things:

1. **Do customers have the information to use a portal?** We sampled
   200 "where is my order" emails. 71% did not quote an order number;
   they identified the order by PO number, by product description or by
   "the one we discussed on Tuesday". A portal keyed on order number
   serves at most 29% of these.

2. **Do these customers want a portal?** We asked eight accounts,
   including both key accounts. Six said they would not use one -
   several explicitly said they email precisely because they want a
   human to confirm. Two said they would use it.

3. **Does one already exist?** Yes. The vendor-supplied customer portal
   has an order status page. Usage over 90 days: 11 distinct customers
   out of 340 active accounts.

## Decision

We will not build a customer-facing status portal.

Instead we will auto-respond to "where is my order" messages with the
current status, resolved from the PO number or the sender's recent
orders where the order number is absent, with a human in the loop for
anything ambiguous. That is tracked as a separate proposal.

## Options considered

### A. Build a new status portal
Rejected. 6-8 weeks. Keyed on data 71% of enquiries do not contain,
duplicating a portal that 3% of customers already ignore.

### B. Improve the existing vendor portal
Rejected. We do not control it, the vendor's change process is
quarterly, and adoption evidence suggests the format is not the problem.
```

```
### C. Email auto-response with status lookup  [direction chosen]
Meets customers in the channel they have chosen. Handles the PO-number
case, which the portal cannot. Estimated 2 weeks.

### D. Do nothing
Rejected: this is the single largest category of inbound volume.

## Consequences

- We are deliberately leaving a "self-service" box unticked. If a future
  stakeholder asks why there is no portal, this document is the answer.
- The auto-response path requires a PO-number-to-order lookup that does
  not currently exist. That is now on the critical path for that work.
- If customer behaviour changes - for example if a large account mandates
  portal use - this should be revisited. The evidence here is from
  Q1 2026 and has a shelf life.

## Evidence

- 200-email sample: analysis/2026-03-po-number-sample.ipynb
- Customer interviews: docs/discovery/account-interviews-2026-03.md
- Portal usage: vendor analytics export, 2026-01-01 to 2026-03-31
```

✓ **The most valuable ADRs are the ones that say no**

A directory full of "we chose X" records is useful. A directory that also contains "we deliberately did not build Y, and here is the evidence" is transformative, because the default assumption about any missing capability is that nobody thought of it. Writing down a considered no protects the decision from being reversed by the next person who has the obvious idea, and it costs twenty minutes.

Keeping ADRs alive

1. **Store them in the repository**, as numbered markdown files in `docs/adr/`, reviewed in pull requests like code. ADRs in a wiki rot; ADRs in the repo get reviewed by people who care.

2. **Never edit an accepted ADR's substance.** Write a new one that supersedes it, and add a "Superseded by ADR-00NN" line to the old one. The historical record is the point.

3. **Write them at the moment of decision**, not retrospectively. A retrospective ADR documents the rationalisation, not the reasoning.

4. **Keep them to one page.**
   If an ADR needs five pages, the decision is actually several decisions and should be split.
5. **Include a "what would change this" section.**
   It converts a historical record into a trigger that someone can watch for.
6. **Link them from the code.**
   A comment saying `// see ADR-0014` above an unusual-looking construct is the cheapest possible documentation of a surprise.
7. **Review the index at handover.**
   Walk the customer's team through the ten most consequential ADRs in person; it is the fastest way to transfer the system's reasoning.

Design Documents and RFCs

A design document exists to get a decision made before code is written, and to make the reasoning reviewable by people who will not read the code. For an FDE it has a second function: it is the artefact that gets you through the customer's architecture and security gates, and writing it in the shape their reviewers expect is the difference between conditional approval in week two and rejection in week eight.

A full worked design document

MARKDOWN

```
# Design: asynchronous warehouse integration for the order pipeline

- Status: In review (decision needed by 2026-03-11)
- Author: A. Krishnan
- Reviewers: T. Berg (architecture), K. Frey (security), S. Ali
- Related: ADR-0014, RB-009, ticket NSD-412

## TL;DR

Move the warehouse call out of the synchronous order path and behind
the platform team's existing managed RabbitMQ. 6 days of work. Expected
Order API p99 drops from 12.3 s to ~320 ms, and the 34-orders-in-90-days
inconsistency class disappears. One decision needed: approval to use the
shared RabbitMQ cluster for customer-order traffic.

## Problem

### What is happening

The Order API performs four steps inside the customer request:
validate, write to the database, notify, then call the warehouse. The
warehouse call is slow and unreliable, and it is the last step.

| Metric | Current | Target |
|---|---|---|
| Order API p50 | 410 ms | 250 ms |
| Order API p95 | 3.1 s | 500 ms |
| Order API p99 | 12.3 s | 2 s |
| Warehouse call p99 | 11 s | n/a (moves off path) |
| Orders stranded in 90 days | 34 | 0 |

### Why it matters

- The customer charter commits to a 2 s order confirmation. We breach
  it on roughly 4% of orders.
- 34 orders in 90 days were committed to our database and then failed
  at the warehouse step. Each required manual reconciliation by the
  order management team, taking roughly 25 minutes each.
- The warehouse vendor's Sunday maintenance window takes our order
  acceptance down with it. This has caused two Monday-morning backlogs
  in the last quarter.

### Why now

The order volume forecast for Q3 is +22%. At current latencies the p99
breach rate grows super-linearly because the warehouse call queues
behind itself under load. We measured this in a load test on 2026-03-04
(results in appendix A).
```

```
## Goals

1. Order acceptance is not blocked by warehouse availability or latency.
2. No order is ever committed to our database without a guaranteed,
   observable path to the warehouse.
3. Order API p99 under 2 s at 2x current peak load.
4. No new vendor, no new operational burden on the customer's team.

## Non-goals

- We are not changing the warehouse API or anything on the vendor side.
- We are not introducing event sourcing or a general eventing platform.
  This is one integration, not a platform migration.
- We are not changing the order data model.
- We are not addressing the notification service's latency (separate,
  smaller problem - ticket NSD-418).

## Proposed design

### Flow

1. Order API validates and writes the order with `status = ACCEPTED`.
2. In the same database transaction, it writes a row to an `outbox`
   table.
3. A relay publishes outbox rows to the `orders` exchange and marks them
   sent. This gives us atomicity between the order write and the
   intent to publish.
4. A warehouse consumer reads `OrderPlaced`, calls the warehouse API,
   and updates `status = CONFIRMED`.
5. Failures retry with exponential backoff; after 5 attempts the message
   goes to `orders.dlq` and an alert fires.

### Why outbox rather than publishing directly

Publishing directly from the API after the database commit introduces a
window where the order exists and the message was never published - the
same class of bug we are trying to remove, just smaller. The outbox
makes the publish intent part of the transaction. Cost: one extra table
and a relay process, roughly a day.

### Message schema

```json
{
  "message_id": "01HQ8F4K2N9VZ...",
  "type": "OrderPlaced",
  "version": 1,
  "occurred_at": "2026-03-14T09:14:22.113Z",
```
```

```
  "order_id": "ORD-2026-0038412",
  "customer_id": "CUST-00291",
  "ship_to": { "...": "..." },
  "lines": [ { "sku": "...", "qty": 4 } ]
}
```

`message_id` is used for idempotency on the consumer side; the warehouse
API is not idempotent, so we maintain a `processed_messages` table and
skip duplicates.

Failure behaviour

Failure	Behaviour	Observable as
RabbitMQ unavailable at publish	Outbox row stays unsent; relay retries every 5 s
Warehouse API 5xx	Retry 5x with backoff (1, 4, 16, 64, 256 s)	`warehouse_retry_t
Warehouse API 4xx	No retry; straight to DLQ	`orders_dlq_depth` gauge, alert at >
Consumer crash mid-call	Message redelivered; `processed_messages` prevents a dupli
Poison message	5 failures then DLQ; never blocks the queue	DLQ alert

Alternatives considered

See ADR-0014 for the full comparison. Summary: in-process thread pool
(loses messages on restart), database outbox with our own poller
(reimplements queue semantics we already have), Kafka (3 weeks, wrong
semantics for a work queue).

Risks

Risk	Likelihood	Impact	Mitigation
Shared RabbitMQ cluster noisy-neighbour	Medium	Medium	Dedicated vhost and a pe
DLQ accumulates unnoticed	Medium	High	Alert at depth > 0, runbook RB-009, name
Customers see "accepted" before warehouse confirms	High	Low	Order status in th
Outbox relay becomes a bottleneck	Low	Medium	Load-tested to 900 msg/s, 8x curr

Security considerations

- Messages contain customer name and ship-to address: personal data
  under GDPR. Message TTL on the queue is 24 hours; DLQ retention is
  7 days, after which messages are dropped and the failure is recorded
  without the payload.
- The RabbitMQ cluster is inside the customer tenancy; no data leaves it.
- Credentials are issued per-service via the existing vault integration;
  no shared credentials.
- Reviewed against the customer's secure design checklist; item-by-item
  response in appendix C.

```
## Rollout plan

| Stage | What | Duration | Rollback |
|---|---|---|---|
| 1 | Deploy consumer and outbox relay; feature flag off. Order API still calls wareho
| 2 | Dual mode: synchronous call continues, outbox also publishes. Consumer logs what
| 3 | Cut over: consumer makes the real call; synchronous path removed behind the flag
| 4 | Remove the synchronous code path. | after 2 weeks stable | Revert commit |

Stage 2 is the important one: it gives us five days of evidence that the
consumer would have made exactly the same warehouse calls, with no
customer risk.

## How we will know it worked

- Order API p99 measured over 7 days post-cutover, compared against the
  same Grafana panel for the 7 days before. Target under 2 s.
- Count of orders in `ACCEPTED` state for longer than 5 minutes. Target
  zero outside a warehouse outage.
- DLQ depth. Target zero; any non-zero value is investigated.

## Open questions

| # | Question | Owner | Needed by |
|---|---|---|---|
| 1 | Can we have a dedicated vhost on the shared cluster? | Platform team | 2026-03-1
| 2 | Is 7-day DLQ retention acceptable to the DPO given the PII content? | K. Frey |
| 3 | Who owns the DLQ alert after handover? | P. Menon | 2026-03-25 |

## Appendices

A. Load test methodology and results
B. Current-state latency breakdown by stage
C. Secure design checklist, item-by-item response
```

What reviewers actually look for

Reviewer	They are checking	What makes them say yes quickly
Architect	Does it fit the estate? Does it create a new pattern?	Explicit reuse of existing platform components; a named non-goal preventing scope sprawl
Security	Data classification, credentials, blast radius, retention	Their checklist answered item by item in an appendix
Operations	Who gets paged, and can they fix it?	A failure table, named alerts, and a runbook reference
Data protection	What personal data, where, for how long	An explicit PII and retention paragraph, not a sentence buried in prose
The sponsor	Cost, time, risk to the date	A TL;DR with effort and a single decision request
The next engineer	Why is it like this?	Alternatives considered, with the reasons for rejection

✦ **Write to the reviewer's checklist, literally**

Ask every gatekeeper for their review criteria before you write — "what do I have to satisfy to get this through?" — and then structure a section or an appendix that answers each criterion in their own wording and order. Reviewers approve documents that are easy to review. This single habit converts multi-round rejections into conditional approvals, and almost nobody does it.

The review process

1. **Circulate a draft early, labelled as a draft.**
   A document marked "Draft — attacking it is the point" gets honest feedback; one marked "for approval" gets silence followed by late objections.

2. **Walk the two hardest reviewers through it individually first.**
   Twenty minutes each, before the formal review. Their objections surface in private where they are cheap.

3. **Set a comment deadline and state what happens after it.**
   "Comments by Friday; I will proceed with the design as written on Monday unless there are open objections."

4. **Resolve comments in the document, visibly.**
Reply to each, and record material changes in a short change log. Reviewers who see their comment addressed engage again next time.

5. **Record the decision and who made it**
, in the document and in the project's decision log.

6. **Write the ADR after approval,**
summarising the decision in one page. The design doc is the full argument; the ADR is the durable record.

Runbooks and Operational Documentation

A runbook is written for a specific reader in a specific state: someone woken at 3 a.m., who did not build the system, who is stressed, and who needs to restore service before understanding anything. Every design decision in a runbook follows from that: imperative numbered steps, exact commands, explicit verification, an escalation path, and absolutely no explanation before the action.

The structure

```
TEXT
```

```
EVERY RUNBOOK HAS EXACTLY THESE SECTIONS, IN THIS ORDER

1. TRIGGER        What alert or symptom brings someone here. Must match
                  the alert name exactly so it can be linked from the page.
2. SEVERITY       What this means for users and how urgently to act.
3. IMPACT         Who is affected and how, in one sentence.
4. PRECONDITIONS  Access, tools and permissions needed before step 1.
                  (Discovering at 3 a.m. that you lack a role is fatal.)
5. DIAGNOSIS      Numbered checks that branch. Each check is one command
                  with an expected result and a "if X go to Y".
6. REMEDIATION    Numbered steps. Exact commands. No placeholders.
7. VERIFICATION   How to confirm it is actually fixed. Non-optional.
8. ROLLBACK       What to do if remediation makes it worse.
9. ESCALATION     Named people, in order, with how to reach them and
                  when to stop trying the previous one.
10. AFTERWARDS    What to record, who to tell, whether a postmortem is
                  needed.
11. BACKGROUND    Why this happens. LAST, because nobody reads it at
                  3 a.m. and everybody reads it at 10 a.m.
```

A worked runbook

MARKDOWN

```
# RB-009: orders dead-letter queue is non-empty

## Trigger

Alert `orders_dlq_depth_nonzero` (Grafana / PagerDuty service
`data-platform`). Fires when `orders.dlq` has one or more messages for
more than 5 minutes.

## Severity

P2. Orders are accepted but not reaching the warehouse. Not customer-
visible immediately; becomes visible when a delivery is missed, which
is typically the next pick wave (every 15 minutes, 06:00–20:00).

## Impact

Each message in the DLQ is one order that the warehouse does not know
about. The customer has received an order confirmation.

## Preconditions

- `kubectl` access to namespace `orders` (role `orders-operator`)
- RabbitMQ management UI access: https://rabbit.internal/#/queues
- Read access to the `orders` Postgres database
- If you do not have these, escalate immediately — do not spend time
  requesting access during an incident.

## Diagnosis

**1. How many messages, and are they still arriving?**

```bash
kubectl exec -n orders deploy/rabbit-tools -- \
  rabbitmqadmin list queues name messages --vhost=orders
```

- `orders.dlq` = 1–5 and stable → likely a single bad message. Go to 2.
- `orders.dlq` growing → systemic. Go to 3.

2. Inspect the messages (does not consume them)

```bash
kubectl exec -n orders deploy/rabbit-tools -- \
  rabbitmqadmin get queue=orders.dlq count=5 ackmode=reject_requeue_true
```

Look at `x-death` in the headers for the rejection reason.
```

````
- `reason: rejected` with HTTP 400 from the warehouse &rarr; bad payload.
  Go to remediation A.
- `reason: expired` &rarr; the message sat longer than the 24 h TTL. The
  warehouse was down for a long period. Go to remediation B.

**3. Is the warehouse API up?**

```bash
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" \
  https://warehouse.vendor.example/api/health
```

- Non-200 or no response &rarr; vendor outage. Go to remediation B and
  notify warehouse ops (see Afterwards).
- 200 &rarr; our consumer is failing. Check consumer logs:

```bash
kubectl logs -n orders deploy/warehouse-consumer --tail=200 | grep ERROR
```

## Remediation

### A. Single bad payload

1. Identify the order id from the message body.
2. Check the order in the database:

   ```sql
   SELECT order_id, status, created_at, ship_to_country
   FROM orders WHERE order_id = 'ORD-...';
   ```

3. If `ship_to_country` is null or not a 2-letter ISO code, this is the
   known defect in issue #211. Fix the record:

   ```sql
   UPDATE orders SET ship_to_country = 'DE'
   WHERE order_id = 'ORD-...' AND ship_to_country IS NULL;
   ```

   Only do this if you can confirm the country from the ship-to address.
   If you cannot, escalate - do not guess.

4. Replay the message:

   ```bash
   kubectl exec -n orders deploy/rabbit-tools -- \
     rabbitmqadmin publish exchange=orders routing_key=order.placed \
     payload="$(cat /tmp/message.json)"
````
````

```

### B. Warehouse outage - bulk replay after recovery

1. Confirm the warehouse is healthy (diagnosis step 3 returns 200).
2. Replay the whole DLQ:

   ```bash
   kubectl create job -n orders --from=cronjob/dlq-replay dlq-replay-manual
   kubectl logs -n orders job/dlq-replay-manual -f
   ```

3. The job replays at 10 messages/second to avoid overwhelming a
   just-recovered warehouse. For more than 500 messages this takes
   over a minute; let it finish.

Verification

All three must be true:

1. DLQ is empty:

   ```bash
   kubectl exec -n orders deploy/rabbit-tools -- \
     rabbitmqadmin list queues name messages --vhost=orders
   ```

2. No orders stuck in ACCEPTED:

   ```sql
   SELECT count(*) FROM orders
   WHERE status = 'ACCEPTED' AND created_at < now() - interval '10 minutes';
   ```

   Expected: 0.

3. The alert has cleared in Grafana (allow 5 minutes).

Rollback

Replaying is idempotent - the consumer checks `processed_messages`
before calling the warehouse, so a duplicate replay does not create a
duplicate shipment. There is nothing to roll back.

If a replay causes the main queue to back up, pause the replay job:

```bash
kubectl delete job -n orders dlq-replay-manual
```
```

```
Messages remain in the DLQ; nothing is lost.

## Escalation

1. Data on-call (PagerDuty `data-platform`) - you are here
2. After 30 minutes unresolved: S. Ali, +49 ... (Signal preferred)
3. After 60 minutes, or any suspicion of duplicate shipments:
   M. Hoeffner, order management, +49 ... - he can halt the pick wave
4. Vendor outage lasting over 2 hours: warehouse ops lead, who owns the
   vendor relationship

## Afterwards

- Record the incident in the incident log with the DLQ count and cause.
- If the cause was a bad payload, add it to issue #211 with the order id.
- If more than 20 messages were affected, or any order was delayed past
  its pick wave, tell M. Hoeffner by 08:00 so customer service can get
  ahead of the calls.
- Postmortem required if: duration over 1 hour, more than 50 orders, or
  any customer-visible delivery impact.

## Background

The DLQ receives messages after 5 failed delivery attempts. The most
common cause by far is a null `ship_to_country`, which the warehouse API
rejects with HTTP 400; this originates from the Hamburg SFTP source file
where the country column is sometimes empty (issue #211, fix scheduled
for the April release). The second cause is a warehouse outage lasting
longer than the retry window of roughly 5 minutes.

Design rationale is in ADR-0014.
```

✗ **The three runbook sins**

First, explaining before acting — the on-call engineer needs step 1 at the top, not three paragraphs of architecture. Second, placeholders: `kubectl logs -n <namespace>` fails at 3 a.m. because the reader does not know the namespace. Third, missing verification — a runbook that ends at "restart the service" leaves the operator unable to tell whether they fixed it or merely moved it. Every remediation needs a check whose result is unambiguous.

Why runbooks rot, and what to do about it

Cause of rot	Symptom	Counter-measure
Commands change with the system	Step 3 fails with "unknown flag"	Reference scripts in the repo rather than inline commands where possible, so they change together
The failure mode is fixed but the runbook stays	A runbook for something that cannot happen any more	Delete it. Add "delete the runbook" to the fix ticket's definition of done
Never used	Nobody discovers the errors in it	Game-day exercises: trigger the failure in a test environment and have someone follow the runbook
Written by the person who never uses it	Assumes knowledge the on-call does not have	Have the actual on-call rota review it, and watch them use it once
No owner	Ambiguity about who updates it	Owner field at the top; reviewed at every postmortem that touches it
Lives outside the repo	Code changes, runbook does not	Keep runbooks in the repo; a PR that changes alerting must touch the runbook

✦ **The game-day test**
Pick a runbook. In a test environment, actually cause the failure. Hand the runbook to someone who has never done this procedure and say nothing while they work. Time it, and write down every point at which they hesitate, guess or ask. That list is your edit. An untested runbook is a hypothesis about what would help, and hypotheses about stressed humans at 3 a.m. are usually wrong.

API and Integration Documentation

API documentation is judged by one metric: time to first successful call. Everything that delays that moment — prose before examples, authentication explained in three places, an error catalogue that omits the error the reader just received — is a defect. The reader is a developer with a deadline, not a student, and they will give you about ten minutes before they ask a human instead.

Examples first, always

TEXT

```
THE ORDER THAT WORKS

1. One complete, runnable example at the very top. Real values. No
   placeholders except the API key, and show exactly what a key looks
   like so the reader can recognise theirs.
2. The response to that example, in full.
3. Authentication setup — how to get a key, where to put it.
4. The endpoints, each with request, response, and errors.
5. Error catalogue.
6. Limits, quotas and versioning.
7. Changelog.

THE ORDER THAT FAILS
1. Introduction and concepts
2. Authentication theory
3. Resource model
4. Endpoint reference tables
5. ... examples somewhere near the bottom, if at all
```

BASH

```bash
# The first thing on the page. It works. Copy, paste, run.

curl -X POST https://api.nordstahl.example/v1/orders \
  -H "Authorization: Bearer nsk_live_4f8a2c91d7e34b06" \
  -H "Content-Type: application/json" \
  -d '{
    "customer_id": "CUST-00291",
    "po_number": "PO-99412",
    "ship_to": {
      "line1": "Industriestrasse 14",
      "city": "Hamburg",
      "postcode": "20095",
      "country": "DE"
    },
    "lines": [
      { "sku": "STL-4410-A", "quantity": 12 },
      { "sku": "STL-2200-B", "quantity": 4 }
    ]
  }'
```

JSON

```
HTTP/1.1 201 Created
Location: /v1/orders/ORD-2026-0038412

{
  "order_id": "ORD-2026-0038412",
  "status": "ACCEPTED",
  "created_at": "2026-03-14T09:14:22.113Z",
  "customer_id": "CUST-00291",
  "po_number": "PO-99412",
  "lines": [
    { "sku": "STL-4410-A", "quantity": 12, "unit_price": "41.20",
      "currency": "EUR", "line_total": "494.40" },
    { "sku": "STL-2200-B", "quantity": 4, "unit_price": "118.00",
      "currency": "EUR", "line_total": "472.00" }
  ],
  "total": "966.40",
  "currency": "EUR",
  "estimated_dispatch": "2026-03-16"
}
```

> **Show the whole response, including the boring fields**
> Truncated examples with `" … ": " … "` cost the reader a round trip to find out what the real shape is. Show every field at least once in a complete example, then use field tables for the detail. The example is the contract most developers will actually code against.

OpenAPI as the source of truth

YAML

```yaml
openapi: 3.1.0
info:
  title: Nordstahl Order API
  version: "1.4.0"
  description: |
    Create and query orders. Orders are accepted synchronously and
    confirmed to the warehouse asynchronously - see the `status` field.
servers:
  - url: https://api.nordstahl.example/v1
    description: Production
  - url: https://api-sandbox.nordstahl.example/v1
    description: Sandbox. Accepts any key beginning nsk_test_.

paths:
  /orders:
    post:
      summary: Create an order
      operationId: createOrder
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateOrderRequest'
            examples:
              minimal:
                summary: Smallest valid request
                value:
                  customer_id: CUST-00291
                  lines: [{ sku: STL-4410-A, quantity: 1 }]
              withPurchaseOrder:
                summary: With a PO number and explicit ship-to
                value:
                  customer_id: CUST-00291
                  po_number: PO-99412
                  ship_to:
                    line1: Industriestrasse 14
                    city: Hamburg
                    postcode: "20095"
                    country: DE
                  lines: [{ sku: STL-4410-A, quantity: 12 }]
      responses:
        "201":
          description: Order accepted. Warehouse confirmation follows.
        "400":
          description: Validation failed. See error catalogue.
        "402":
          description: Customer is over their credit limit.
```

```yaml
        "409":
          description: Duplicate - this po_number already exists for
            this customer within the last 30 days.
        "422":
          description: One or more SKUs are unknown or discontinued.
        "429":
          description: Rate limited. Retry after the Retry-After header.

components:
  schemas:
    CreateOrderRequest:
      type: object
      required: [customer_id, lines]
      properties:
        customer_id:
          type: string
          pattern: '^CUST-[0-9]{5}$'
          description: Your account identifier. Visible in the portal
            under Account > Details.
        po_number:
          type: string
          maxLength: 32
          description: Your purchase order reference. Optional, but
            strongly recommended - it is the only identifier our
            customer-service team can search on.
        ship_to:
          $ref: '#/components/schemas/Address'
          description: Omit to use the default ship-to on the account.
        lines:
          type: array
          minItems: 1
          maxItems: 200
          items:
            $ref: '#/components/schemas/OrderLine'
```

✦ **Descriptions are where documentation lives**

Most OpenAPI files describe types and nothing else: `customer_id: string`. The value is entirely in the description field — where the reader gets the value, what it is for, what happens if it is absent, and what the constraint means in business terms. "Optional, but strongly recommended — it is the only identifier our customer-service team can search on" is the sentence that prevents four hundred support tickets.

The error catalogue

The error catalogue is the part of API documentation that developers actually read, because they arrive at it after something has gone wrong. It must be searchable by the exact string the API returned.

HTTP	code	Meaning	Retry?	What to d
400	`invalid_request`	Body is not valid JSON, or a required field is missing	No	See `detai` offending f
400	`invalid_country_code`	`ship_to.country` is not a 2-letter ISO 3166-1 code	No	Use "DE", r "DEU"
401	`missing_credentials`	No Authorization header	No	Send `Auth` `nsk_...`
401	`invalid_key`	Key is malformed, revoked or for the wrong environment	No	Sandbox k `nsk_test_` in producti
402	`credit_limit_exceeded`	Order total would exceed the account's credit limit	No	Contact yo manager; `details.a` the headro
403	`scope_denied`	The key is valid but lacks `orders:write`	No	Generate a scope in th
404	`order_not_found`	Unknown order id, or it belongs to another customer	No	Deliberatel to avoid le
409	`duplicate_po_number`	Same `po_number` for this customer in the last 30 days	No	`details.e` gives the o idempoten an error in
422	`unknown_sku`	One or more SKUs do not exist	No	`details.s` ones
422	`sku_discontinued`	SKU exists but is no longer orderable	No	`details.r` where one

429	`rate_limited`	Over 100 requests/minute for this key	Yes	Honour `Re` (seconds). I
500	`internal_error`	Our fault	Yes	Retry with I `request_` support.
503	`maintenance`	Planned maintenance window	Yes	Retry after announced the change

JSON

```
// Every error has the same shape. Document it once, prominently.
{
  "error": {
    "code": "unknown_sku",
    "message": "2 of 3 SKUs could not be found.",
    "details": {
      "skus": ["STL-9999-X", "STL-0000-Z"]
    },
    "request_id": "req_01HQ8F4K2N9VZ",
    "documentation_url": "https://docs.nordstahl.example/errors#unknown_sku"
  }
}
```

⚠ **The three things every error must have**

A stable machine-readable `code` that never changes wording (clients will switch on it), a `request_id` that correlates to your logs so support conversations take one minute instead of twenty, and a link to the specific documentation anchor for that error. Messages are for humans and may change; codes are an API contract and may not.

Authentication, quotas and versioning

TEXT

```
AUTHENTICATION - what to document

 - How to get a key: exact click path in the portal, and who can do it
   (role required). Most "auth docs" skip this and it is the first
   blocker.
 - What a key looks like, by environment: nsk_live_ ...  vs nsk_test_ ...
 - Where it goes: header name, scheme, exact format
 - Scopes: the full list, what each permits, and how to change them
 - Rotation: how to rotate without downtime (issue new, deploy, revoke
   old) and the maximum overlap period
 - What to do if a key leaks: the revoke endpoint and who to tell

QUOTAS - state all four
 - The limit:        100 requests per minute per key
 - The window type:  sliding, not fixed - bursts do not reset on the minute
 - The response:     429 with Retry-After in seconds
 - The headers on EVERY response, so clients can self-throttle:
       X-RateLimit-Limit: 100
       X-RateLimit-Remaining: 43
       X-RateLimit-Reset: 1773483262
 - What is NOT counted: 4xx validation failures still count; health
   checks do not. Say which, because clients will ask.

VERSIONING - answer these five, explicitly
 1. Where does the version live?       URL path: /v1/
 2. What counts as breaking?           Removing a field, changing a type,
                                       adding a required request field,
                                       changing an error code, tightening
                                       validation
 3. What is NOT breaking?              Adding an optional request field,
                                       adding a response field, adding a
                                       new error code for a new condition,
                                       adding an enum value
 4. How long is a version supported?   24 months from the successor's GA
 5. How are you told?                  Changelog, Sunset header on
                                       deprecated endpoints, and an email
                                       to the technical contact 180, 90
                                       and 30 days out

>> Point 3 is the one everyone forgets and it matters most: tell clients
   what they must tolerate, so that an additive change is not treated as
   an outage. "Clients MUST ignore unknown fields" belongs in the
   documentation, not in a tribal assumption.
```

Integration documentation beyond the API

- [ ] A sequence diagram of the full happy path, including the asynchronous parts the reader cannot see from the API alone.
- [ ] An explicit statement of consistency: "the order is ACCEPTED immediately and CONFIRMED typically within 60 seconds; do not treat ACCEPTED as dispatched."
- [ ] Idempotency: what key, what window, what happens on a repeat. This is the single most common integration bug.
- [ ] Sandbox: how it differs from production, which behaviours are simulated, and how to trigger each error deliberately.
- [ ] A test SKU, test customer and test card equivalent, so an integrator can exercise every path without real data.
- [ ] Webhooks if any: payload shape, retry policy, signature verification with a worked example, and how to replay.
- [ ] A "common mistakes" section built from real support tickets, in the words the tickets used.

The Handover Pack

The handover pack is the most commercially valuable artefact a Forward Deployed Engineer produces. Everything else is consumed during the engagement; this is what the customer still has in three years. It is also the thing most often thrown together in the last week, from memory, by someone whose next engagement has already started. Build it incrementally from week two and the final week becomes a review rather than a panic.

The full table of contents

TEXT

HANDOVER PACK - order pipeline - Nordstahl Distribution
Owner after 2026-04-11: Nordstahl data team
Accepted by: _________________ Date: ________

1. EXECUTIVE SUMMARY [1 page]
   What was built, why, what it achieves, what it cost, what remains.
   Written for someone who was not involved. No jargon, one diagram.
   CONTAINS: the measured before/after, the scope boundary, the
   support model in two sentences, and who owns it now.

2. ARCHITECTURE OVERVIEW [2-3 pages]
   A C4 context diagram and a container diagram, each with prose.
   Every component: what it does, what it talks to, where the code is,
   where it runs.
   CONTAINS: the deliberate design decisions and a link to each ADR.
   DOES NOT CONTAIN: implementation detail that will drift.

3. ENVIRONMENTS
   For each of dev / test / prod:
     - URLs and hostnames
     - Cloud subscription / account / project ids
     - Kubernetes cluster, namespace
     - Database host, name, and how to connect
     - What differs from production (data volumes, external stubs)
     - How to deploy to it, and who is allowed to
     - Cost per month and which cost centre it bills to
   >> The cost line gets forgotten and then someone's budget is
      surprised in month four.

4. ACCESS AND CREDENTIALS INVENTORY
   NOT the credentials themselves. The inventory:
     - Every secret the system uses, by name
     - Where it is stored (vault path), who can read it
     - What it is for, what breaks without it
     - Expiry / rotation date and the rotation procedure
     - Who currently holds personal access that should be revoked
   >> THE KEY QUESTION THIS SECTION ANSWERS: "the FDE's account is
      disabled tomorrow - what stops working?" Answer it explicitly,
      in a named subsection, and then make the answer "nothing".

Secret	Where	Used by	Rotate	Owner
sftp-privkey	vault:/orders/sftp	ingest job	2027-01-15	data team
warehouse-api-key	vault:/orders/wh	consumer	180 days	data team
rabbit-orders-user	vault:/orders/mq	api, consumer	365 days	platform
grafana-sa-token	vault:/obs/orders	dashboards	365 days	platform

5. RUNBOOKS [index]

```
   One per known failure mode, linked. Table: alert name → runbook →
   severity → owner. If an alert exists with no runbook, say so
   explicitly rather than leaving the gap silent.

6. MONITORING AND ALERTING
   - Dashboards: what each shows, what normal looks like (with numbers)
   - Alerts: name, condition, severity, routing, runbook link
   - SLOs: what is promised, measured how, current attainment
   - What is NOT monitored, and the risk that represents
   >> The "what is not monitored" subsection is the most honest and most
      useful page in the whole pack.

7. KNOWN ISSUES AND ACCEPTED LIMITATIONS
   Each with: what it is, who it affects, frequency, workaround,
   ticket reference, and whether it is accepted or merely unfixed.
   Distinguish "we decided this is fine" from "we ran out of time".

8. DEFERRED WORK AND TECHNICAL DEBT
   Each item: what, why deferred, estimated effort, consequence of
   continuing to defer, and a suggested trigger for doing it.
   >> This section converts "the consultants left a mess" into "here is
      a costed backlog we agreed together". Write it honestly; including
      the items that embarrass you.

9. DATA
   - Sources: where each dataset comes from, who owns it upstream
   - Schemas: tables, key fields, where the DDL lives
   - Lineage: a diagram of what derives from what
   - Retention: what is kept, how long, and what deletes it
   - PII: exactly which fields, in which stores, under what basis
   - Backups: what, how often, how to restore, when it was last tested
   >> "When was the restore last tested" is the question auditors ask
      and teams cannot answer. Test it before handover and record
      the date.

10. SUPPORT MODEL
    - First line: who, hours, what they can resolve
    - Second line: who, how reached, SLA
    - Escalation: named people in order, with contact methods
    - Out of hours: what is covered and what explicitly is not
    - Vendor support: contract references, portal links, entitlements
    - What to do about the thing the model does not cover

11. CONTACTS
    Everyone involved, their role, what they know, and how to reach
    them. Include people who have left the project but hold context.
    | Name | Role | Knows about | Contact |

12. DECISIONS AND CHANGE HISTORY
```

```
    The decision log and the ADR index. Not a narrative – a table with
    dates, decisions and who made them.

13. HOW TO MAKE THE THREE MOST LIKELY CHANGES
    Predict what they will need to change first and write the how-to now:
      – "How to add a new SFTP source"
      – "How to change the batch size safely"
      – "How to add a field to the OrderPlaced message"
    >> This is the section that most distinguishes a good handover.
       You know what they will need; they do not know yet.

14. BENEFIT MEASUREMENT
    The agreed baseline, the exact query, the result, and how to
    re-run it. So that in twelve months someone can check whether the
    benefit persisted.

APPENDICES
    A. Glossary of project-specific terms
    B. Third-party dependencies, versions, licences, EOL dates
    C. Compliance artefacts: security review outcome, DPIA reference
```

The acceptance test for a handover

TEXT

```
NOT "is the document written". The test is behavioural.

SETUP
  Pick a named operator from the customer's team who was not on the
  project. Pick three realistic scenarios - at least one failure
  scenario, one routine operation, one change.

RUN
  They work through each scenario using only the written material.
  You sit in the room and say NOTHING. Not a hint, not a nudge.
  A second person records every hesitation, wrong turn and question.

SCENARIOS (example)
  1. "The DLQ alert has fired at 02:00. Resolve it."
  2. "Add the Rotterdam SFTP source. It uses the same format."
  3. "Rotate the warehouse API key without downtime."

PASS CRITERIA
  All three completed unaided. Zero questions to you.

WHAT YOU DO WITH THE FAILURES
  Every hesitation is a documentation defect. Fix it in the reader's
  own words - if they said "I didn't know where the vault path was",
  the fix is a vault path in that step, not a general improvement to
  the credentials section.
  Then repeat with a different operator.

PUT IT IN THE SOW
  Acceptance criterion: "A Nordstahl-nominated operator completes three
  nominated scenarios using only the handover documentation, observed,
  without assistance." This phrasing makes the deliverable their
  capability, not your document - which is the actual thing being
  bought.
```

✓ **The six-week question**

Ask yourself weekly, from week two: "if I became unreachable for six weeks starting tomorrow, what would break and who would be stuck?" Whatever you answer is the next thing to write. Done consistently, the handover pack assembles itself and the final week contains no surprises. Done only at the end, you will discover in week nineteen that half the knowledge you need to write down has already faded.

Diagrams

A diagram is a compression device: it carries structural information that prose conveys badly and tables cannot convey at all. It is also the artefact most likely to be wrong, because diagrams are drawn once and never revisited. Choose the right type for the question being asked, keep them in version control as code, and label them so they can be read without you standing next to them.

Choosing the right diagram

Reader's question	Diagram	Keep it to
"What is this system and who uses it?"	C4 context	The system as one box, plus users and external systems
"What are the deployable pieces?"	C4 container	Apps, databases, queues — things that run
"What is inside this service?"	C4 component	Only for complex services; usually skip
"What happens, in what order?"	Sequence	One scenario per diagram, 5–9 participants
"What are the decision branches?"	Flowchart	One process, under 15 nodes
"What states can this be in?"	State diagram	Entities with a lifecycle: orders, tickets, accounts
"Where does it physically run?"	Deployment	Regions, clusters, networks, trust boundaries
"How is the data shaped?"	ER diagram	Key entities and cardinalities only, not every column
"What derives from what?"	Lineage / DAG	Datasets and the jobs between them

C4, briefly and practically

TEXT

```
LEVEL 1  CONTEXT     Who uses the system, what it talks to.
                     Audience: everyone, including non-technical.
                     YOU ALWAYS NEED THIS ONE.

LEVEL 2  CONTAINER   The deployable units: web app, API, consumer,
                     database, queue. Technology labelled on each.
                     Audience: engineers, architects, operations.
                     YOU ALMOST ALWAYS NEED THIS ONE.

LEVEL 3  COMPONENT   The major pieces inside one container.
                     Audience: engineers working in that container.
                     USUALLY SKIP. It goes stale fastest and the code
                     answers the same question more accurately.

LEVEL 4  CODE        Class diagrams. Do not draw these. Generate if
                     ever needed.

THE RULE OF THUMB FOR AN FDE HANDOVER: one context diagram and one
container diagram, both maintained as code, both in the repo. That is
90% of the value for 20% of the maintenance.
```

Diagrams as code — Mermaid

TEXT

```
graph TB
    subgraph External
        CUST[Customer<br/>places orders]
        WH[Warehouse system<br/>vendor SaaS]
        SAP[SAP ERP<br/>master data]
    end

    subgraph "Order Pipeline (this system)"
        API[Order API<br/>Python/FastAPI]
        DB[(Postgres<br/>orders)]
        MQ{{RabbitMQ<br/>orders exchange}}
        CONS[Warehouse consumer<br/>Python]
        RELAY[Outbox relay<br/>Python]
    end

    CUST ⟶|HTTPS POST /v1/orders| API
    API ⟶|write order + outbox row<br/>one transaction| DB
    RELAY ⟶|poll unsent| DB
    RELAY ⟶|publish OrderPlaced| MQ
    MQ ⟶|consume| CONS
    CONS ⟶|POST /shipments| WH
    CONS ⟶|update status| DB
    API ⟶|read customer, credit| SAP

    classDef ext fill:#eff4ff,stroke:#6b7688
    class CUST,WH,SAP ext
```

TEXT

```
sequenceDiagram
    autonumber
    actor C as Customer
    participant A as Order API
    participant D as Postgres
    participant R as Outbox relay
    participant Q as RabbitMQ
    participant W as Warehouse consumer
    participant V as Warehouse (vendor)

    C->>A: POST /v1/orders
    A->>D: BEGIN; INSERT order (ACCEPTED); INSERT outbox; COMMIT
    A-->>C: 201 Created {status: ACCEPTED}
    Note over C,A: Customer is done in ~250 ms

    R->>D: SELECT * FROM outbox WHERE sent_at IS NULL
    R->>Q: publish OrderPlaced
    R->>D: UPDATE outbox SET sent_at = now()

    Q->>W: deliver OrderPlaced
    W->>D: SELECT 1 FROM processed_messages WHERE id = ?
    alt already processed
        W-->>Q: ack (skip)
    else new message
        W->>V: POST /shipments
        alt 2xx
            V-->>W: 201
            W->>D: UPDATE order SET status = CONFIRMED
            W-->>Q: ack
        else 5xx or timeout
            V-->>W: error
            W-->>Q: nack (retry, up to 5x)
            Note over W,Q: after 5 failures the message<br/>goes to orders.dlq and ale
        end
    end
```

TEXT

```
stateDiagram-v2
    [*] ⟶ ACCEPTED: POST /v1/orders succeeds
    ACCEPTED ⟶ CONFIRMED: warehouse accepts
    ACCEPTED ⟶ FAILED: warehouse rejects (4xx)
    ACCEPTED ⟶ ACCEPTED: retry in progress
    CONFIRMED ⟶ DISPATCHED: pick wave completes
    DISPATCHED ⟶ DELIVERED: carrier confirms
    FAILED ⟶ ACCEPTED: manual replay from DLQ
    CONFIRMED ⟶ CANCELLED: cancelled before dispatch
    ACCEPTED ⟶ CANCELLED: cancelled before confirmation
    DELIVERED ⟶ [*]
    CANCELLED ⟶ [*]

    note right of ACCEPTED
        Customer sees "Order received".
        Warehouse does not know yet.
        Normal duration under 60 s.
    end note
```

PlantUML for deployment and trust boundaries

TEXT

```
@startuml
!define RECTANGLE class

node "Customer network" {
  [Customer ERP] as ERP
}

cloud "Internet" {
}

node "Nordstahl Azure tenancy (westeurope)" {
  node "AKS cluster: nsd-prod" {
    package "namespace: orders" {
      [Order API] as API
      [Outbox relay] as RELAY
      [Warehouse consumer] as CONS
    }
  }
  database "Azure Postgres\nFlexible Server" as PG
  queue "RabbitMQ\n(platform managed)" as MQ
  [Key Vault] as KV
}

node "Vendor SaaS (eu-central-1)" {
  [Warehouse API] as WH
}

ERP ⟶ API : HTTPS, mTLS\nclient cert per customer
API ⟶ PG : private endpoint
RELAY ⟶ PG
RELAY ⟶ MQ
MQ ⟶ CONS
CONS ⟶ PG
CONS ⟶ WH : HTTPS, API key\nvia NAT gateway, static egress IP
API ..> KV : managed identity
CONS ..> KV : managed identity

note right of WH
  Trust boundary.
  Personal data (name, ship-to)
  leaves our tenancy here.
  Covered by DPA ref NSD-DPA-2024-03.
end note
@enduml
```

Why diagrams-as-code wins

A PNG exported from a drawing tool is reviewed by nobody, updated by one person who still has the source file, and wrong within a quarter. Mermaid or PlantUML source sits next to the code, appears as a diff in pull requests, can be required to change when the architecture changes, and renders natively in GitHub, GitLab and most wikis. The visual polish is lower and the accuracy is far higher, and accuracy is the only property that matters.

Labelling conventions

- [ ] Label every arrow with the interaction and, where useful, the protocol: "publishes OrderPlaced (AMQP)", not a bare line.
- [ ] State direction unambiguously. If the arrow means "calls", say so; if it means "data flows", say that instead — they are often opposite.
- [ ] Put the technology on each box: "Postgres 16", "Python/FastAPI". A future reader needs it and it costs four characters.
- [ ] Mark trust boundaries and anything that leaves your control, explicitly. Security reviewers look for this first.
- [ ] Use the same name for a component as the repository, the Kubernetes deployment and the dashboard. Synonyms across artefacts cost hours.
- [ ] Date the diagram and name its owner, in the caption or the file header.
- [ ] Write a caption that states the diagram's claim, not its subject: "Order acceptance does not depend on warehouse availability" beats "Order pipeline architecture".

Diagram anti-patterns

Anti-pattern	Why it fails	Fix
The everything diagram	40 boxes, every system, unreadable at any zoom	Split by level and by question; three diagrams beat one
Unlabelled arrows	The reader cannot tell a call from a data flow from a dependency	Label every edge
Mixed abstraction levels	A class next to a cloud region	One level per diagram, stated in the caption
Decorative icons	Vendor logos convey nothing and age badly	Text labels
Colour as meaning, with no key	Only the author knows what blue means	Add a legend, or do not use colour semantically
Screenshot of a whiteboard	Unsearchable, unmaintainable, often illegible	Acceptable for 48 hours; then redraw as code
No happy-path highlight	The main flow is indistinguishable from error paths	Draw the happy path prominently; put errors in a second diagram
Diagram without prose	Readers infer different things from the same picture	Always pair a diagram with a paragraph stating what it shows

Release Notes, Changelogs and User-Facing Comms

Release notes are the only documentation some of your users will ever read, and they are written by engineers for engineers almost everywhere. The discipline is to separate three different documents that are routinely conflated: a changelog for people integrating with you, release notes for people using the product, and a migration guide for people who must change something because of you.

Three documents, three readers

	Changelog	Release notes	Migration guide
Reader	Integrating developer	End user, operator, buyer	Someone whose code will break
Question answered	"What changed in the contract?"	"What can I do now that I could not?"	"What must I do, and by when?"
Style	Terse, complete, categorised	Benefit-led, plain language	Imperative steps with before/after
Includes	Every change, including internal where it affects behaviour	Only what a user would notice	Only breaking changes
Ordering	Reverse chronological, by version	Most valuable first	In the order the work must be done

A changelog that works

MARKDOWN

```
# Changelog

All notable changes to the Order API. This project follows semantic
versioning. Dates are release dates in UTC.

Breaking changes appear only in major versions and are announced at
least 180 days in advance via the technical contact on your account.

## [1.4.0] - 2026-03-28

### Added
- `po_number` field on `POST /v1/orders` (optional, max 32 chars).
  Enables lookup by purchase order in `GET /v1/orders?po_number=`.
- `estimated_dispatch` in the create-order response. Best-effort, based
  on current stock and pick-wave schedule; not a commitment.
- New error code `duplicate_po_number` (HTTP 409) when the same
  `po_number` is submitted twice for one customer within 30 days.
  Returns `details.existing_order_id`.

### Changed
- `status` may now be `ACCEPTED` at the moment of response. Previously
  the response was only returned after warehouse confirmation.
  **This changes timing, not the contract.** Orders reach `CONFIRMED`
  typically within 60 seconds. If your integration treats a 201 as
  "dispatched", review it - see the note below.
- p99 response time for `POST /v1/orders` improved from 12.3 s to
  320 ms as a result of the above.

### Deprecated
- `GET /v1/orders/{id}/warehouse-status`. Use the `status` field on the
  order itself. The endpoint continues to work until 2027-03-28 and
  returns a `Sunset` header.

### Fixed
- Orders with a null `ship_to.country` were accepted and then failed
  silently downstream. They are now rejected at the API with
  `invalid_country_code` (HTTP 400). (#211)
- `X-RateLimit-Remaining` was off by one at the start of each window.

### Security
- API keys are now scoped. Existing keys were granted all scopes
  automatically; no action required. New keys default to `orders:read`.

### Internal (no API impact, listed for operators)
- Warehouse calls now run in a separate consumer with a dead-letter
  queue. Operators: see runbook RB-009 for the new DLQ alert.

---
```

```
## [1.3.2] - 2026-02-14

### Fixed
- Timezone handling in `created_at` for the `Europe/Berlin` region.
  Timestamps were correct in storage but rendered one hour off in the
  response during CET. Clients that parsed the ISO-8601 offset were
  unaffected; clients that assumed UTC were not.
```

> ⚠ **The "this changes timing, not the contract" note**
>
> That paragraph is the most important thing in the 1.4.0 entry. Technically no field changed shape, so semantic versioning says it is a minor release — but any integrator who treated a 201 as "the warehouse has it" is now broken in a way no schema diff would reveal. Behavioural changes that are not schema changes are the most dangerous category, and they must be called out explicitly with "review your integration if you assumed X".

Release notes for a non-technical reader

TEXT

```
BEFORE — written by an engineer

  Release 1.4.0
  - Implemented asynchronous warehouse integration via AMQP with an
    outbox pattern for transactional guarantees
  - Added po_number to the order schema with a unique constraint over
    a 30-day window
  - Refactored the error handling in the ingest path
  - Bumped dependencies

AFTER — written for the people who use it

  What's new — 28 March

  Orders now confirm in under a second
  Placing an order used to take up to 12 seconds at busy times because
  we waited for the warehouse to respond. We now confirm your order
  immediately and pass it to the warehouse in the background, usually
  within a minute. You will see the order status move from "Received"
  to "Confirmed with warehouse".

  You can now search by your own PO number
  Add your purchase order number when you place an order and you — and
  our customer service team — can find it by that number later. This
  is the number our team will ask you for, so adding it saves a
  conversation.

  Orders with an incomplete delivery address are now caught immediately
  Previously an order with a missing country could be accepted and then
  fail later without a clear message. These are now rejected straight
  away with an explanation of what is missing.

  One thing to be aware of
  Because orders now confirm before the warehouse has them, "Received"
  no longer means "being picked". Wait for "Confirmed with warehouse"
  before assuming a dispatch date. In normal operation this takes under
  a minute.
```

A migration guide

MARKDOWN

````
# Migrating from v1 to v2 of the Order API

**Who this affects:** anyone calling `POST /v1/orders` or
`GET /v1/orders`. If you only consume webhooks, no action is needed.

**Deadline:** v1 is supported until **2027-09-30**. After that date v1
endpoints return HTTP 410.

**Estimated effort:** 2–4 hours for a typical integration.

## Summary of breaking changes

| # | Change | Action |
|---|---|---|
| 1 | `total` is now an object, not a string | Read `total.amount` and `total.currency
| 2 | `lines[].unit_price` removed | Use `lines[].price.amount` |
| 3 | `status` values renamed | Map old to new (table below) |
| 4 | `GET /orders` is cursor-paginated | Replace offset paging |

## 1. Money is now an object

**Before**
```json
{ "total": "966.40", "currency": "EUR" }
```

**After**
```json
{ "total": { "amount": "966.40", "currency": "EUR" } }
```

**Why:** line items and order totals could have different currencies in
multi-currency accounts, and a single top-level `currency` field could
not express that.

**Do this:** replace `order.total` with `order.total.amount` and
`order.currency` with `order.total.currency`. The string format of
`amount` is unchanged — a decimal string, never a float.

## 3. Status values renamed

| v1 value | v2 value | Notes |
|---|---|---|
| `PENDING` | `ACCEPTED` | Same meaning |
| `PROCESSING` | `CONFIRMED` | Same meaning |
| `COMPLETE` | `DELIVERED` | Same meaning |
| `ERROR` | `FAILED` | Same meaning |
| – | `CANCELLED` | **New.** Previously a cancelled order was `ERROR`. If you treat `E
````

```
## How to migrate safely

1. **Run both in parallel.** v1 and v2 serve the same orders; you can
   call v2 read endpoints against existing orders today.
2. **Compare for one week.** Call both, log any differences. We provide
   a comparison helper: `https://docs…/tools/v1v2-diff`.
3. **Switch writes last.** Move `GET` calls to v2 first; they are
   read-only and reversible.
4. **Tell us when you have finished** so we can stop sending reminders:
   reply to the migration email or set `migrated: true` in the portal.

## Getting help

- Sandbox: `https://api-sandbox.nordstahl.example/v2`, any `nsk_test_` key
- Questions: integration-support@nordstahl.example, 2 business days
- If the deadline is a problem, contact your account manager before
  2027-06-30. Extensions are possible but must be agreed in advance.
```

Writing for non-native English readers

A large share of your readers will be reading in their second or third language, often quickly, often under pressure. The adjustments are small and they make the text better for native readers too.

Avoid	Because	Use
Idioms: "out of the box", "under the hood", "a ballpark figure"	Not learnable from context; frequently mistranslated	"by default", "internally", "an approximate figure"
Phrasal verbs: "spin up", "roll out", "tear down", "kick off"	The meaning is not derivable from the parts	"start", "release", "delete", "begin"
Negative questions: "Don't you want to keep the old value?"	"Yes" and "no" mean opposite things in different languages	"Do you want to keep the old value?"
Double negatives: "not uncommon", "it is not impossible"	Doubles the parsing load, halves the clarity	"common", "possible"
Long noun stacks: "order status update notification service"	Ambiguous grouping; which noun modifies which?	"the service that notifies you when an order status changes"
Humour, sarcasm, cultural references	Does not survive translation; can cause offence	Nothing. Delete it.
Ambiguous pronouns: "it", "this", "that" across sentences	Referent is unclear when parsing is effortful	Repeat the noun: "the consumer", not "it"
Contractions in formal docs: "won't", "can't", "it's"	"It's" vs "its" is a known stumbling block	"will not", "cannot", "it is"
Sentences over 25 words	Subordinate clauses compound the load	Split at the comma

✦ **Write once, translate cheaply**
If anything you write may be translated — and in a multinational enterprise it probably will be — the choices above cut cost and error rates dramatically. Additionally: keep sentences self-contained so they can be translated as units; never build a sentence from concatenated string fragments in code; avoid embedding numbers or names mid-sentence where word order differs across languages; and never rely on text inside images, which cannot be translated at all without redrawing.

Localisation considerations for an FDE

- ☐ UI strings live in resource files from day one, even if there is only one language today. Retrofitting is five times the work.

- [ ] Never concatenate: `"Deleted " + n + " orders"` breaks in languages with different plural rules and word order. Use a full parameterised template per plural form.
- [ ] Dates: ISO-8601 in logs, APIs and anything machine-read; locale-formatted only in user-facing output. "03/04/2026" is two different days depending on the reader.
- [ ] Numbers: decimal comma versus point, and thousands separators, differ across Europe. Render per locale; store and transmit unambiguously.
- [ ] Names and addresses: do not assume a first name, a last name, a postcode format, or that a state field exists.
- [ ] Allow for text expansion: German runs 30% longer than English, so fixed-width labels will truncate.
- [ ] Timezones: store UTC, display local, and always label which you are showing. "02:00" with no zone has caused more incidents than any single bug class.

Email, Chat and Async Communication

An FDE writes more words in email and chat than in every formal document combined, and those words are where most decisions actually get made and lost. The craft is the same as long-form — conclusion first, specific asks, named owners — compressed into a form people read on a phone between meetings.

The subject line does half the work

Weak	Strong
Update	Order pipeline week 6 — AMBER — one decision needed by Friday
Quick question	Need the SFTP host for the Rotterdam feed — blocking me today
Re: Re: Re: Fwd: project	Decision: using RabbitMQ not Kafka — summary and reasoning
Meeting notes	Steering 18 Mar — 3 decisions taken, 1 action for Rachel
Following up	Still need Finance IT capacity — 4 days of float left
Design doc	Design review: async warehouse integration — comments by Tue 11th

A good subject line states the topic, the status and the ask. People triage by subject on a phone; a subject that does not say whether action is required gets opened last.

The three-paragraph rule

TEXT

```
PARAGRAPH 1   The ask or the conclusion. What you need, from whom, when.
PARAGRAPH 2   The minimum context required to act on paragraph 1.
PARAGRAPH 3   Anything else, or a link to it.

If you need a fourth paragraph, you are writing a document. Write the
document, attach it, and keep the email to three paragraphs that explain
why they should open it.

WORKED EXAMPLE

Subject: Need a decision on queue choice by Thursday - 10 min read

Priya - I need you to pick between two options by Thursday so the build
can start Monday. My recommendation is option C (managed RabbitMQ):
6 days, no new vendor, uses infrastructure the platform team already
runs.

The alternative worth considering is option B (database outbox with our
own poller): 4 days and no new infrastructure, but we would be building
queue mechanics that already exist elsewhere in your estate, and we
would own them after I leave. The full comparison is in the design doc,
section "Alternatives considered" - about 10 minutes.

If I do not hear by Thursday 17:00 I will proceed with C and record it
as a default decision; C is the one that is easiest to reverse.

Arun
```

When not to write

Situation	Why writing fails	Do instead
Bad news with real impact	Tone is unreadable; the recipient fills the gaps badly	Call, then send the written version within the hour
A disagreement that has gone two rounds	Written argument hardens positions and creates a record of conflict	Fifteen minutes on a call; then write the agreed outcome
Anything emotional	You cannot take it back and it will be forwarded	Wait a night. Then usually have a conversation instead.
A decision needing real-time back-and-forth	Asynchronous rounds take days for something that takes ten minutes	A short call with a written summary after
Something requiring more than three rounds of clarification	You are using the wrong medium	Call. Then write the conclusion.
Criticism of a named person	Written criticism outlives its context and can be weaponised	Say it in person; write only the process observation

⚠ **Assume everything you write will be forwarded**

Not out of paranoia — out of accuracy. Emails to a sponsor get forwarded to their boss. Chat messages get screenshotted into other channels. The practical rule: never write anything about a person that you would not say to them, never write a half-formed opinion as though it were an assessment, and never put a number in writing that you cannot defend. This costs nothing and prevents the single most common career-damaging mistake in consulting.

Chat that respects other people

TEXT

```
BAD - the drip (four notifications, no information in any of them)
  09:02  "hey"
  09:02  "quick question"
  09:03  "are you around?"
  09:07  "it's about the SFTP thing"

GOOD - one message, complete, answerable asynchronously
  09:02  "@anita - do you know the host for the Rotterdam SFTP feed?
          I have the credentials from the vault but no hostname, and
          it is blocking me today. If it is easier, point me at who
          would know. No rush before this afternoon."

BAD - the context-free escalation
  "the pipeline is broken"

GOOD
  "orders pipeline: DLQ has 47 messages since 08:10, all failing with
   invalid_country_code. Looks like the Hamburg file again (#211).
   I am following RB-009 and will update in 20 minutes. No action
   needed from anyone yet - flagging in case customer service get
   calls about Hamburg orders."

BAD - the decision buried in a thread
  [47-message thread, decision made at message 31, nobody can find it]

GOOD
  After the thread resolves, post:
  "DECISION (thread above): we will normalise 'Str.' to 'Strasse'
   before comparison. Owner: me. Recorded as D-011 in the decision
   log. Any objection, say so today; after that it is settled."
  Then pin it, and put it in the decision log.
```

Meeting notes and the decision log

TEXT

```
MEETING NOTE - send within 2 hours, always

Subject: Steering 18 Mar - 3 decisions, 1 action for Rachel

Present: Rachel (COO), Priya, Thomas, Sanjay, Arun
Apologies: Klaus (sent comments in advance, incorporated)

DECISIONS
 D-009  Phase 2 scope: include returns automation, defer phone channel.
        Decided by: Rachel. Effective immediately.
 D-010  Use the shared RabbitMQ cluster with a dedicated vhost.
        Decided by: Thomas, with Klaus's written agreement.
 D-011  Aggregate-only metrics; no individual productivity reporting.
        Decided by: Rachel. To be confirmed in writing to the works
        council by Priya.

ACTIONS
 [ ] Rachel to secure 2 days of Finance IT capacity before 7 Mar
 [ ] Priya to write to the works council confirming D-011, by 25 Mar
 [ ] Arun to circulate the phase 2 estimate by 25 Mar

DISCUSSED, NOT DECIDED
 - Whether the Austrian subsidiary joins phase 2 or phase 3.
   Rachel will discuss with the AT country manager; back on 1 Apr.

NEXT MEETING  15 Apr, 09:00. I will send the pack on 13 Apr.

If anything above does not match your recollection, tell me today
and I will correct it. Otherwise this is the record.

--
WHY THAT LAST LINE MATTERS: it converts your notes into the agreed
version of events by default, and gives people a cheap way to correct
you while memories are fresh. Almost nobody objects; the ones who do,
you wanted to hear from anyway.
```

✓ **Decisions deserve their own home**

Decisions buried in meeting notes are effectively lost, because nobody searches twelve months of notes. Maintain a single append-only decision log — id, date, decision, who made it, status, link to the context — shared with the customer. Never edit a past row; add a superseding row that references the old id. In month nine, when someone says "we never agreed to that", this file ends the conversation in ten seconds and without anyone having to remember anything.

Writing with AI Assistance

Language models are excellent at structure, tedious transformation and first drafts, and unreliable at anything requiring knowledge of your specific system. Used well they roughly double writing throughput; used badly they produce fluent, confident, plausible documentation that is wrong in ways reviewers do not catch because it reads so well. The discipline is knowing exactly which parts of the job can be delegated.

What to delegate and what never to

SAFE TO DELEGATE

Restructuring content you wrote into BLUF order. Generating a first draft from your bullet points. Converting a design doc into an ADR. Producing a table from prose. Tightening sentences. Generating a glossary from a document. Drafting alternative phrasings of a difficult paragraph. Writing the boring parts of an OpenAPI description once the schema exists. Suggesting what a reviewer might object to.

NEVER DELEGATE

Any factual claim about your system — numbers, field names, endpoints, behaviour, limits. Root-cause analysis in an incident. Security or data-protection statements. Commitments, dates, estimates. The judgement of what to include or omit. Anything that goes to a customer unreviewed. Nuanced interpersonal communication, where generic warmth reads as insincerity.

Prompts that produce useful output

TEXT

```
WEAK   "Write documentation for my API."
       → generic, invented endpoints, wrong error codes, confident tone

STRONG "Here is my OpenAPI spec and three real request/response pairs
        captured from the sandbox. Write the 'Getting started' page as a
        how-to guide in the Diataxis sense: a complete runnable curl
        example first, then the response in full, then authentication
        setup. Do not invent any field, status code or behaviour that is
        not in the material I gave you. If something needed for the page
        is missing from my material, list it at the end under MISSING
        rather than filling the gap."

---

WEAK   "Make this better."

STRONG "Rewrite this status update in BLUF order. The reader is a
        non-technical sponsor who has 90 seconds. Put the two blockers
        and the named asks above everything else. Keep every number
        exactly as written. Cut at least 30% of the words. Do not add
        any claim that is not in my original."

---

WEAK   "Review my design doc."

STRONG "You are a sceptical enterprise security architect reviewing this
        design. List, in priority order, the questions you would ask and
        the objections you would raise before approving it. Focus on data
        classification, credential handling, blast radius and retention.
        Do not suggest improvements - only list what you would challenge."

---

WEAK   "Write a runbook for this alert."

STRONG "Here is the alert definition, the consumer source, and a
        transcript of how we resolved this manually last Tuesday. Draft a
        runbook with these sections in this order: trigger, severity,
        impact, preconditions, diagnosis (branching numbered checks),
        remediation, verification, rollback, escalation, afterwards,
        background. Every command must come from the transcript or the
        source - mark anything you are unsure of with TODO-VERIFY."
```

The review loop

1 **You supply the facts; the model supplies the shape.**

Never ask for content you have not provided in some form. The generation step should be a transformation, not a retrieval.

2. **Require an explicit gaps list.**
"List anything you needed and did not have" converts silent invention into a visible to-do.

3. **Verify every specific.**
Field names against the schema, commands against a shell, numbers against the source, links by clicking them. Fluency is not evidence.

4. **Read it aloud for voice.**
Model prose has tells: "robust", "seamless", "leverage", "it's important to note that", tricolons everywhere, and a relentless evenness of tone. Edit them out or your documentation stops sounding like a person who knows the system.

5. **Cut 30% after generation.**
Models over-explain and restate. The editing checklist applies more strictly to generated text, not less.

6. **Never paste customer data into a tool that is not covered by the agreement.**
This is a contractual and often legal matter, not a preference. Check before the first use, not after.

✕ **The specific failure mode to watch for**

A model will produce a beautifully formatted runbook with an entirely plausible `kubectl` command that references a namespace which does not exist, a flag that was removed two versions ago, or a metric name it inferred from context. It reads exactly like the nine correct commands around it. Reviewers skim it because the document is well structured. At 3 a.m. the operator runs it and it fails, or worse, succeeds against the wrong thing. Every command in generated operational documentation must be executed once, by you, before it ships.

Keeping your voice

TEXT

```
TELLS OF UNEDITED MODEL PROSE — search for these before publishing

  "It's important to note that …"          → delete the phrase entirely
  "robust", "seamless", "cutting-edge"     → name the actual property
  "leverage", "utilise", "facilitate"      → use, use, help
  "In today's fast-paced environment"      → delete the sentence
  "This ensures that …"                    → often an unverified claim
  "Whether you're X or Y, …"               → delete; pick one audience
  "Let's dive in" / "Let's explore"        → delete
  Three-item lists everywhere              → vary; some things have 2 or 5
  Every paragraph the same length          → vary deliberately
  A summary paragraph at the end of every  → keep one, delete the rest
    section restating the section

WHAT GOOD HUMAN TECHNICAL PROSE HAS THAT GENERATED PROSE LACKS
  - Specific numbers from your own measurement
  - A named person, system or date
  - An admission of what you do not know
  - An opinion with a reason: "I recommend X because Y"
  - The odd short sentence. For emphasis.
  - Reference to something that happened: "when this failed in March"
```

Docs as a Product

Documentation that nobody owns, nobody reviews and nobody measures decays into a liability — an archive of confidently wrong statements that new joiners trust. Treating docs as a product means they have an owner, a review process, a quality bar enforced by tooling, a release process, and a deletion process. The last of those is the one almost everyone skips.

Docs as code

TEXT

```
THE MODEL

  Source       Markdown (or reStructuredText / AsciiDoc) in the same
               repository as the code it describes
  Review       Pull requests, with the same reviewers as the code
  Build        A static site generator in CI (MkDocs, Docusaurus,
               Sphinx, Hugo)
  Publish      Automatically on merge to main
  Version      Docs are branched and tagged with the code, so v1.3 docs
               describe v1.3 behaviour
  Test         Link checking, linting, and executing code samples

WHY IT BEATS A WIKI
  - The diff is visible and reviewable
  - Changing behaviour and changing its documentation is one commit
  - You can require documentation changes in a PR template
  - Stale pages are discoverable by file age, not by hope
  - It survives the wiki migration that happens every three years

WHAT A WIKI IS STILL GOOD FOR
  - Meeting notes, decision logs, onboarding checklists, anything that
    is organisational rather than about the code
  - Content edited by people who will not use git, which in an
    enterprise is most people. Do not fight this; choose per audience.
```

YAML

```yaml
# .github/workflows/docs.yml — the minimum useful docs CI
name: docs
on:
  pull_request:
    paths: ["docs/**", "mkdocs.yml", "**/*.md"]
  push:
    branches: [main]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Lint prose with Vale
        uses: errata-ai/vale-action@reviewdog
        with:
          files: docs
          fail_on_error: true

      - name: Check links
        run: |
          npx --yes lychee --no-progress --exclude-mail \
            'docs/**/*.md' 'README.md'

      - name: Verify code samples run
        run: |
          # extracts ```bash blocks tagged "verify" and executes them
          ./scripts/test-docs-samples.sh

      - name: Build site
        run: mkdocs build --strict   # --strict fails on broken refs
```

Linting prose with Vale

YAML

```
# .vale.ini
StylesPath = .vale/styles
MinAlertLevel = suggestion

Packages = Microsoft, write-good

[*.md]
BasedOnStyles = Vale, Microsoft, write-good, Nordstahl

# Microsoft style is opinionated about contractions; we allow them
# in release notes but not in reference documentation.
Microsoft.Contractions = NO

[docs/reference/*.md]
Microsoft.Contractions = error

[docs/runbooks/*.md]
# Runbooks must be imperative and must not hedge.
Nordstahl.NoHedging = error
Nordstahl.NoPlaceholders = error
```

YAML

```yaml
# .vale/styles/Nordstahl/NoHedging.yml
extends: existence
message: "Remove the hedge '%s' - say what is true, or state the condition."
level: error
ignorecase: true
tokens:
  - basically
  - essentially
  - simply
  - just
  - obviously
  - of course
  - it should be noted
  - please note that
  - somewhat
  - fairly
  - quite
---

# .vale/styles/Nordstahl/NoPlaceholders.yml
extends: existence
message: "Placeholder '%s' found. Runbook commands must be runnable."
level: error
raw:
  - '<your-[a-z-]+>'
  - '<insert[^>]*>'
  - 'TODO'
  - 'TBD'
  - 'XXX'
---

# .vale/styles/Nordstahl/Terms.yml
extends: substitution
message: "Use '%s' instead of '%s' - keep terminology consistent."
level: warning
ignorecase: true
swap:
  "job|task": job
  "message queue|broker": queue
  "customer id|client id|account id": customer_id
  "web hook|web-hook": webhook
  "e-mail": email
```

✦ **Start the linter at warning level**

Turning on a full Microsoft or Google style pack at error level against an existing documentation set produces four thousand failures and a team that disables the linter. Start with warnings, fix opportunistically, and promote individual rules to error as they reach zero. The two rules worth promoting first are "no placeholders in runbooks" and a terminology substitution list — both catch real, costly defects rather than style preferences.

Style guides

	Microsoft Writing Style Guide	Google developer documentation style guide
Tone	Warm, conversational, contractions encouraged	Neutral, precise, slightly more formal
Strength	Excellent on UI text, error messages, accessibility and inclusive language	Excellent on API reference, code samples, formatting conventions
Vale package	`Microsoft`	`Google`
Best for	Product and end-user documentation	Developer-facing reference and guides
Watch out	The conversational register can read as flippant in operational docs	Prescriptive on details that may not matter to you

Pick one, write a one-page local supplement for your project-specific terminology and exceptions, and never debate it again. The specific choice matters far less than the consistency; the debate costs more than either option.

TEXT

```
LOCAL STYLE SUPPLEMENT - one page, in the repo, linked from CONTRIBUTING

We follow the Google developer documentation style guide, with these
project-specific rules.

TERMINOLOGY (enforced by Vale)
  job          not task, not process
  queue        not broker, not message bus
  customer_id  not client id, not account number
  consumer     the warehouse consumer service specifically
  order        a customer order; never used for an internal job

VOICE
  Use "you" for the reader. Use "we" only for decisions the team made.
  Never use "the user" in developer-facing docs - say "you".

NUMBERS
  Always with units and a basis. "p95 320 ms at 100 rps", never "fast".
  Use thin spaces, not commas, in code: 100000 not 100,000.

COMMANDS
  Never use placeholders. Use real, working example values. If a value
  is secret, show its shape: nsk_live_4f8a2c91d7e34b06.

DATES
  ISO-8601 everywhere in docs: 2026-03-14. Never 03/14/26.

HEADINGS
  Sentence case. Informative, not categorical.
  "Configure the warehouse consumer" not "Configuration"

RUNBOOKS
  Imperative mood. Numbered. One action per step. Every remediation
  followed by a verification step. No background before step 1.
```

Measuring whether documentation works

Signal	How to get it	What it tells you
Time to first successful run	Watch a new joiner; time it	Whether the quickstart works. The single best metric.
Questions asked in chat	Tag support questions that were answerable from docs	Each one is a documentation defect with a free bug report attached
Page views and exit pages	Site analytics	High traffic plus high exit usually means the page failed
Site search terms with no results	Search logs	The exact words readers use for things you named differently
Runbook follow-through	Game days; incident reviews	Whether operational docs survive contact with stress
Doc age versus code age	`git log` per file	Pages untouched while their code changed are probably wrong
"Was this helpful?"	A widget on each page	Weak signal alone; useful when paired with a free-text box
Onboarding time	Days until a new joiner's first merged change	The aggregate outcome documentation is trying to improve

> **The question log**
> Keep a single file where every question anyone asks you about the system gets recorded verbatim, with the date. Review it monthly. Each repeated question is a documentation defect, and the reader's own wording tells you both where the answer belongs and what to call it. This is the cheapest documentation research available and it requires no tooling — just the discipline of pasting the question before you answer it.

Killing stale documentation

1. **Put a review date and an owner in the front matter of every page.**
   No owner means no page.

2. **Run a quarterly report of pages past their review date**
   and pages whose code has changed since the doc did. Both are candidates.

3. **Default to deletion, not revision.**
   The question is "would we write this today?", not "is anything in it still true". Most stale pages should go.

4. **Archive rather than agonise.**
   Move it to an `archive/` directory with a banner stating the date it was last known accurate and that it is unmaintained. Nothing is lost and nobody is misled.

5. **Make deletion part of the fix.**
   When a failure mode is designed out, deleting its runbook belongs in the definition of done for that ticket.

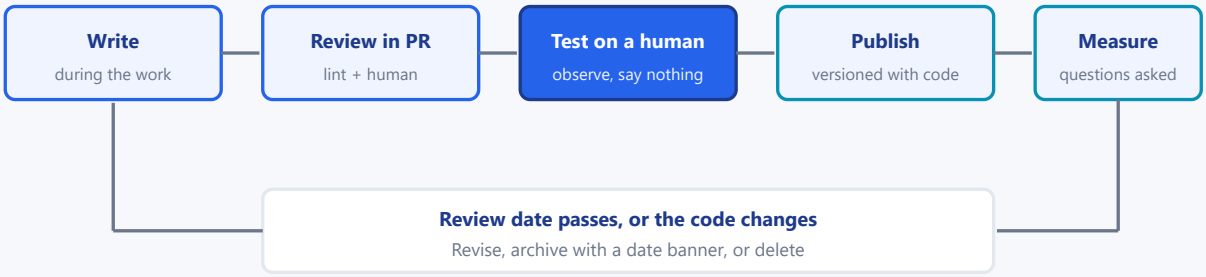
6. **Never leave a page that is half right.**
   A page with three accurate sections and one that is silently wrong is more dangerous than no page, because the accurate parts earn trust that the wrong part then exploits.

YAML

```yaml
---

# front matter on every page - cheap, and it makes staleness measurable
title: Configure the warehouse consumer
owner: data-platform
reviewed: 2026-03-28
review_every: 180d
applies_to: "≥1.4.0"
status: current        # current | deprecated | archived
---
```

> ⚠ **The archive banner**
> When you archive a page, put this at the top and nowhere else: "This page was last verified on 2026-03-28 against version 1.4.0. It is no longer maintained and may be wrong. For current behaviour see [link]." Readers can then use it as a historical reference without being misled, which is exactly the right amount of trust to grant an old document.



Documentation has a lifecycle, and the step everyone omits is the third one: watching a real person try to use it.

Interview Questions

Writing questions in engineering interviews are usually probing for three things: whether you have an audience model, whether you understand that documentation is a product with maintenance costs, and whether you have actually been responsible for knowledge surviving your departure. Answer with structure, a specific artefact and a measurable test.

> **How do you decide what to document and what to leave out?**
>
> I start from the reader and what they must do afterwards, because that determines everything else. Concretely I ask two questions: who reads this, and what decision or action follows? If I cannot name a reader, I do not write the document. Then I apply a filter — would removing this sentence change what the reader does? If not, it goes. The things I always document regardless of appetite are the ones that cannot be recovered from code: why a decision was made and what was rejected, what the system deliberately does not do, how to operate it when it fails, and what breaks if a specific person is unavailable. The things I deliberately do not document are anything the code expresses better, anything that will drift faster than it will be read — component-level internals are the classic example — and anything I would be writing for completeness rather than for a reader. I also use the six-week test continuously during an engagement: if I were unreachable for six weeks starting tomorrow, what would break and who would be stuck? Whatever I answer is the next thing I write, which means the handover pack assembles itself rather than being improvised in the final week.

> **Explain the Diátaxis framework and why it matters.**
>
> It separates documentation into four types along two axes — whether the reader is learning or working, and whether they need theoretical knowledge or practical steps. That yields tutorials for beginners learning by doing, how-to guides for competent people with a specific goal, reference for people who need a fact while working, and explanation for people who want to understand why. It matters because most bad documentation is not badly written; it is two or three of these blended into

one file, which fails all of them. A tutorial that pauses to explain the consensus protocol loses the beginner. A how-to that starts by installing a dev environment wastes the time of someone who already has a running system. Reference with recommendations embedded in it becomes unscannable and ages badly. The practical value is that "where does this paragraph go?" stops being a matter of taste and becomes answerable: ask whether the reader is learning or working and whether they are acquiring or applying knowledge, and you get a quadrant. In my own work the most common correction is pulling explanation out of runbooks — the background about why a failure happens belongs at the very bottom, because nobody reads it at 3 a.m. and everybody reads it at 10 a.m.

What makes a good runbook?

A runbook is written for one specific reader: someone woken at 3 a.m., who did not build the system, who is stressed, and who needs to restore service before understanding anything. Everything follows from that. It opens with the trigger — matching the alert name exactly so it can be linked from the page — then severity and impact in one line each, then preconditions, because discovering at 3 a.m. that you lack a Kubernetes role is fatal. Then branching numbered diagnosis where each check is one command with an expected result and an explicit "if X go to Y", then remediation with exact runnable commands and no placeholders, then verification, rollback, escalation with named people in order, and what to record afterwards. Background goes last, never first. The three sins I watch for are explaining before acting, placeholders like a bracketed namespace that the reader cannot fill in, and a missing verification step — a runbook ending at "restart the service" leaves the operator unable to tell whether they fixed it or moved it. The way I validate one is a game day: cause the failure in a test environment, hand the runbook to someone who has never done the procedure, and sit silently recording every hesitation. Every hesitation is an edit. An untested runbook is a hypothesis about stressed humans, and those hypotheses are usually wrong.

Walk me through how you would write a design document for a significant change.

I lead with a TL;DR that states the recommendation, the effort and the single decision I need, because the sponsor may read nothing else. Then the problem with measured numbers — current versus target in a table, and specifically why it matters now, which is usually a forecast, a date or a breach of something they committed to. Then goals and, importantly, non-goals, which prevent the document being read as a platform migration when it is one integration. Then the proposed design, including a failure-behaviour table: for each failure, what happens, and what it is observable as. Then alternatives considered with the reason each was rejected, which is the section future readers value most. Then risks with mitigations, a security and data-protection section, a staged rollout with the rollback for each stage, and an explicit "how we will know it worked" with the metric and measurement window. Finally open questions with owners and dates. The thing I do that most changes outcomes is asking each gatekeeper up front what their review criteria are and then answering those criteria item by item, in their wording, in an appendix. Reviewers approve documents that are easy to review, and almost nobody does this — it converts multi-round rejections into conditional approvals.

How do you write for an executive versus for an engineer?

Same facts, different documents, and I write them separately rather than trying to serve both. The executive has about ninety seconds and needs to know whether to act and whether they are exposed, so the first paragraph is the conclusion, the number and the decision required, with no mechanism at all. For an incident I would write "revenue figures were overstated by €412,000, corrected, nothing external used them, root cause fixed, no decision needed from you — flagging in case you want to brief the CFO." The engineering version of the same incident is a timeline with UTC timestamps, the three compounding causes, the fixes with owners and dates, and the generalised lesson, because that audience needs to prevent the class of bug rather than assess exposure. The operator version is different again: an alert-triggered runbook with imperative numbered steps, exact queries, a verification check and a branch for "what if month-end has already run". Attempting one document for all three produces something useless to each. The counter-intuitive part is that writing three documents is faster than writing one merged one, because each is shorter and the structure tells you what belongs where.

How do you stop documentation going stale?

Four mechanisms, and the last one is the one teams skip. First, docs live in the repository next to the code, reviewed in pull requests, so that changing behaviour and changing its description is one commit and one review. Second, every page has front matter with an owner and a review date, which makes staleness measurable — I run a quarterly report of pages past their review date and pages whose code has changed since the doc did. Third, automation in CI: link checking, a prose linter, building with strict mode so broken references fail, and executing tagged code samples so that a command that stopped working fails the build rather than failing an operator. Fourth, and most importantly, deletion. The default question for a stale page is "would we write this today?", not "is anything still true", and most stale pages should simply go. When a failure mode is designed out, deleting its runbook belongs in the definition of done for that ticket. I archive rather than agonise — move it with a banner stating the date it was last verified and that it is unmaintained — because a page that is three-quarters right is more dangerous than no page: the accurate parts earn trust that the wrong part then exploits.

What goes in an ADR, and why bother?

One page: context including the constraints that applied at the time, the decision, the options considered with the reason each was rejected, the consequences including the negative ones, and a section on what would change this decision. The reason to bother is that the most expensive missing information in any inherited system is why — why this database, why this boundary, why this odd constraint everyone is now afraid to remove. Without a record, every new senior engineer re-opens the same three debates. With one, the conversation takes ninety seconds. The rejected options are usually more valuable than the chosen one, because they encode the constraints: "we rejected Kafka because we needed work-queue semantics, not a log, and neither ordering nor replay is a requirement" saves a future team two weeks. The most valuable ADRs are the ones that record a no — a deliberate decision not to build something, with the evidence — because the default assumption about any missing capability is that nobody thought of it. Operationally: store them numbered in the

repo, never edit an accepted one, write a superseding one instead, keep them to a page, and reference them from the code with a comment above the construct they explain.

How do you get people to actually read what you write?

Mostly by accepting that they will not read it and designing for that. The conclusion goes in the first two sentences, so the ninety-second reader gets the most important thing. Headings are informative rather than categorical — "Recommendation: RabbitMQ, 6 days, no new vendor" rather than "Conclusion" — so that the table of contents alone is a summary. I apply the skim test: read only the title, headings and first sentence of each paragraph, and if that is not a coherent complete summary, the structure is wrong. I put the ask above the fold with a name and a date on it, because a document with no clear ask gets filed. I keep paragraphs to three to six lines and put more than three numbers in a table rather than a sentence. For email specifically, the subject line states topic, status and whether action is needed, because people triage on a phone. And I give readers an exit — "if you only need the decision, stop after section 2" — which sounds like it reduces readership and actually increases it, because people trust a document that respects their time and come back to it.

Tell me about documentation you wrote that failed.

I wrote a handover pack for a data pipeline that was, by any document-quality measure, good: architecture, environments, runbooks, data lineage, support model. Three weeks after I left, the team called me because an overnight job had failed and nobody could restart it. The runbook said to re-run the job, which was correct, but it assumed a Kubernetes role that only two people had and that I had not listed anywhere, because I had that access and never thought about it. The failure was that I had tested the document by reading it rather than by watching someone use it. What I changed permanently is the acceptance test: I now pick an operator who was not on the project, pick three realistic scenarios including at least one failure, and sit in the room silently while they work through them using only the written material, with a second person recording every hesitation. Every hesitation is a documentation defect and I fix it in the reader's own words — if they said "I didn't know where the vault path was", the fix is the vault path in that step, not a general improvement to the credentials section. I also now put that behavioural test into the statement of work as the acceptance criterion, phrased as their operator succeeding rather than my document existing, which makes the deliverable their capability rather than my prose.

How do you use AI tools for writing without losing accuracy?

I treat it as a transformation tool, not a retrieval tool: I supply the facts and it supplies the shape. So I will give it my OpenAPI spec and three real captured request-response pairs and ask for a getting-started page in a specified structure, with an explicit instruction not to invent any field, status code or behaviour that is not in the material, and to list anything it needed and did not have under a MISSING heading. That last instruction is the important one, because it converts silent invention into a visible to-do list. What I never delegate is any factual claim about the system, root-cause analysis during an incident, security or data-protection statements, commitments and dates, and the judgement of what to include or omit. The specific failure mode I guard against is a beautifully formatted runbook containing a plausible command with a namespace that does not exist or a flag removed two versions

ago — it reads exactly like the correct commands around it, reviewers skim it because the document is well structured, and it fails at 3 a.m. So every command in generated operational documentation gets executed once by me before it ships. I also edit for voice, because model prose has tells — "robust", "seamless", "it's important to note that", every paragraph the same length — and documentation that does not sound like a person who knows the system is trusted less, correctly.

What is the most valuable thing you can leave behind at the end of an engagement?

The handover pack, and specifically the parts of it that are uncomfortable to write. Architecture and runbooks are table stakes. The sections that actually determine whether the customer succeeds without me are the credentials and access inventory — which answers "my account is disabled tomorrow, what stops working?" explicitly — the known-issues list that distinguishes "we decided this is fine" from "we ran out of time", the deferred-work register with honest estimates including the items that embarrass me, the "what is not monitored" subsection, and a section I call "how to make the three most likely changes", where I predict what they will need to change first and write the how-to before they know they need it. That last one is what most distinguishes a good handover, because I know what they will hit and they do not yet. Underneath all of it, the deferred-work register is the artefact that converts "the consultants left a mess" into "here is a costed backlog we agreed together", which is a completely different conversation and the one that gets you invited back. And the test of the whole thing is behavioural, not documentary: their own team demonstrating the system to another part of the business, correctly, without me present.

How would you improve the documentation of a system you have just joined?

I would start by measuring rather than rewriting, because the instinct to immediately restructure everything produces a beautiful set of documents that solve the previous author's problems rather than the readers'. First, the newcomer test on myself: I follow the README from a clean machine, time it, and write down every point where I had to ask someone or open another file. That list is the first backlog and it is free. Second, I start a question log — every question anyone asks me about the system, pasted verbatim with a date, before I answer it — and review it monthly; repeated questions are documentation defects with the reader's own wording attached, which tells me both where the answer belongs and what to call it. Third, a staleness audit: compare the last-modified date of each doc against the code it describes, and default to deleting rather than revising anything that would not be written today. Fourth, I check whether the four document types are blended — usually the README is doing the job of reference, tutorial and explanation at once — and split rather than polish. Only then would I write anything new, and the first new thing would be whatever the six-week test says: if the person who currently holds the context were unavailable for six weeks, what breaks? That is always a more valuable page than the one I would have written from instinct.

Writing Drills and a 30-Day Improvement Plan

Writing improves through repetition with feedback, not through reading about writing. The drills below take between ten and sixty minutes, use work you already have, and each produces an artefact you can show someone. The feedback loop matters more than the volume: a page rewritten after watching someone misread it teaches more than ten pages written alone.

Daily drills

Drill	What you do	Time
BLUF one thing	Take one message you were about to send and move the conclusion to sentence one	2 min
Cut 30%	Take any paragraph you wrote and remove 30% of the words without losing information	5 min
Subject line	Write the subject line before the email, stating topic, status and whether action is needed	1 min
Kill a hedge	Find one hedged sentence and replace it with precision about conditions	2 min
Number it	Find one vague quantifier in your writing — "most", "soon", "fast" — and replace it with a measured value	3 min
Question log	Paste the next question anyone asks you, verbatim, into a file before you answer it	1 min
Decision post	After any thread that resolves something, post a one-paragraph "DECISION:" summary and pin it	3 min

Weekly drills

1. **Three audiences.**
   Take one thing that happened this week and write it three ways: for an executive in 90 seconds, for an engineer who must prevent recurrence, for an operator who must handle it next time. Compare what survived into each.

2. **The skim test.**

Take your longest recent document. Read only the title, headings and first sentences. If it is not a coherent summary, rewrite the headings to be informative and reorder.

3. **Write one ADR**
for a decision already made in your current work, including the options you rejected and what would change the decision. Twenty minutes. Show it to someone who was in the room and ask what you got wrong.

4. **Run the newcomer test.**
Hand your repo to someone who has not seen it, say nothing, and record every question. Each question is a README defect; fix them in their words.

5. **Game-day a runbook.**
Cause a failure in a test environment and have someone else follow the runbook while you stay silent. Edit every hesitation.

6. **Rewrite a sentence pair.**
Find the worst sentence you wrote this week and produce a before/after. Keep a file of these; after a month the pattern in your own failures becomes obvious.

7. **Diagram one thing as code.**
Convert a whiteboard photo or a drawing-tool export into Mermaid, commit it, and label every arrow.

The 30-day plan

TEXT

```
WEEK 1 — STRUCTURE
 Goal: stop burying the conclusion.
 Mon  Rewrite your last three status updates in BLUF order. Compare
      word counts and note how much was chronology carrying no load.
 Tue  Take your longest document. Run the skim test. Rewrite every
      heading to say something rather than name a category.
 Wed  Write the same event three ways: executive, engineer, operator.
      Give all three to a colleague and ask which they would want.
 Thu  Apply the ruthless editing checklist to one real document.
      Count the words removed. Target 30%.
 Fri  Review the week's emails. For each, could the subject line have
      told the reader whether action was needed? Rewrite the worst five.
 DAILY: conclusion in sentence one, every message, no exceptions.
 ARTEFACTS: 3 rewritten updates, 1 restructured document, 3 versions
 of one event.

WEEK 2 — PRECISION
 Goal: never write an unverifiable claim again.
 Mon  Find every hedge in one document — may, could, should, somewhat,
      basically, essentially. Delete or replace with a condition.
 Tue  Find every vague quantifier and replace with a number, a unit and
      a source. Where you cannot, say so explicitly instead of hedging.
 Wed  Build your own before/after file. Write 10 pairs from your real
      writing, with a one-line reason for each. This becomes your
      personal style guide.
 Thu  Rewrite one set of error messages or log lines so that each one
      tells the reader what happened, what to do, and what identifier
      to quote.
 Fri  Write one complete failure-behaviour table for something you own:
      failure, behaviour, observable as.
 ARTEFACTS: 10 before/after pairs, 1 failure table, 1 de-hedged document.

WEEK 3 — THE DURABLE ARTEFACTS
 Goal: produce the things that survive you.
 Mon  Write an ADR for a decision already made. Include rejected options
      and "what would change this decision".
 Tue  Write a second ADR — this one recording something you decided NOT
      to build, with the evidence.
 Wed  Write or rewrite one runbook using the 11-section structure. No
      placeholders. Every command runnable.
 Thu  Game-day it. Someone else follows it while you say nothing.
      Record every hesitation; edit in their words.
 Fri  Convert one diagram to Mermaid or PlantUML, commit it next to the
      code, label every arrow, and write the caption as a claim.
 ARTEFACTS: 2 ADRs, 1 tested runbook, 1 diagram as code.

WEEK 4 — THE SYSTEM
 Goal: make good writing the default rather than an effort.
```

```
Mon  Run the newcomer test on your main repo. Record every question.
Tue  Fix every README defect from Monday, in the reader's own words.
     Add the troubleshooting entries verbatim.
Wed  Set up docs CI: a prose linter at warning level, link checking,
     and a strict build. Promote "no placeholders in runbooks" to error.
Thu  Write the one-page local style supplement: terminology, voice,
     numbers, commands, dates, headings. Link it from CONTRIBUTING.
Fri  Draft the handover pack table of contents for your current work
     and answer honestly: if you were unreachable for six weeks
     starting tomorrow, what breaks? Write the first missing page.
ARTEFACTS: fixed README, docs CI, style supplement, handover TOC,
one page that did not exist before and should have.
```

Self-assessment at day 30

- [ ] Every document I wrote this month states its conclusion in the first two sentences.
- [ ] My headings pass the table-of-contents test: reading them alone is a coherent summary.
- [ ] I have a before/after file of my own sentences and I can name my two most common faults.
- [ ] I have written at least one ADR recording a deliberate decision not to build something.
- [ ] At least one of my runbooks has been followed, start to finish, by someone who did not write it.
- [ ] My README has been tested on a newcomer who had zero questions.
- [ ] I have a question log and I have fixed at least three documentation defects it surfaced.
- [ ] Every page I own has an owner and a review date in its front matter.
- [ ] I have deleted or archived at least one stale page rather than revising it.
- [ ] I can answer the six-week question for my current work, and the answer is "nothing".

✓ **The habit that compounds fastest**

If you adopt only one thing from this guide, make it the question log: before answering any question about your system, paste the question verbatim into a file with the date. It costs ten seconds, requires no tooling, and within a month it will have told you exactly which pages to write, what to call things in the words your readers actually use, and which of your existing documents are failing. Every other improvement in this guide gets easier once you are working from evidence about real readers instead of from your own assumptions about them.